CS 135 — Designing Functional Programs

Vishesh Gupta

University of Waterloo

Contents

1 Introduction to CS 135 and Racket **1**
- 1.1 What this course is about . . . 1
- 1.2 From mathematical functions to Racket . . . 1
  - 1.2.1 Numbers . . . 2
- 1.3 Expressions, constants and functions . . . 2
- 1.4 Syntax and semantics . . . 2
  - 1.4.1 Syntax . . . 2
  - 1.4.2 Semantics: the substitution model . . . 3
  - 1.4.3 Run-time errors . . . 3
- 1.5 Identifiers and scope (preview) . . . 4
- 1.6 Programs in DrRacket . . . 4
- 1.7 Common pitfalls . . . 4
- 1.8 Summary . . . 4

2 The Design Recipe **5**
- 2.1 Why a recipe? . . . 5
- 2.2 The components in detail . . . 6
  - 2.2.1 Purpose . . . 6
  - 2.2.2 Contract . . . 6
  - 2.2.3 Examples . . . 6
  - 2.2.4 Tests . . . 6
- 2.3 Data definitions and templates . . . 7
- 2.4 A complete example . . . 7
- 2.5 Style conventions . . . 7
- 2.6 Common pitfalls . . . 8
- 2.7 Summary . . . 8

3 Simple Data **9**
- 3.1 Booleans and predicates . . . 9
  - 3.1.1 `and`, `or`, `not` . . . 9
- 3.2 Conditional expressions . . . 10
  - 3.2.1 Simplifying conditionals . . . 11
  - 3.2.2 Testing conditionals . . . 11
- 3.3 Symbols . . . 11
- 3.4 Strings . . . 11
- 3.5 Characters . . . 12
- 3.6 Mixed data and type predicates . . . 12

3.7 Summary . . . 12

4 Structures **13**
4.1 Defining a structure . . . 13
4.1.1 Substitution rules for structures . . . 13
4.2 Data definitions and structure definitions . . . 14
4.3 Templates for structures . . . 14
4.4 Nested structures . . . 15
4.5 Mixed data with structures . . . 15
4.6 Common pitfalls . . . 16
4.7 Summary . . . 16

5 Lists **17**
5.1 Constructing lists . . . 17
5.2 Deconstructing lists . . . 17
5.3 Data definition for lists . . . 18
5.4 The list template . . . 18
5.5 First recursive functions . . . 18
5.6 List abbreviations . . . 19
5.7 Lists of structures and strings . . . 20
5.8 Common pitfalls . . . 20
5.9 Summary . . . 21

6 Working with Recursion on Lists **22**
6.1 Functions producing Booleans . . . 22
6.2 Functions producing lists . . . 22
6.2.1 Transforming each element (a “map” pattern) . . . 22
6.2.2 Selecting some elements (a “filter” pattern) . . . 23
6.2.3 Appending . . . 23
6.3 Non-empty lists . . . 23
6.4 Insertion sort . . . 23
6.5 Lists of lists . . . 24
6.6 Wrapper functions . . . 25
6.7 Testing recursive functions . . . 25
6.8 Summary . . . 25

7 Types of Recursion **26**
7.1 Accumulative recursion . . . 26
7.1.1 Motivation: reversing a list . . . 26
7.1.2 Template . . . 27
7.1.3 Maximum with an accumulator . . . 27
7.1.4 Average in one pass . . . 27
7.1.5 Context-dependent processing . . . 28
7.2 Mutual recursion . . . 28
7.3 Generative recursion (preview) . . . 29
7.4 Choosing a kind of recursion . . . 29
7.5 Summary . . . 29

8 Natural Numbers and Processing Multiple Lists 30
8.1 Natural numbers as recursive data . . . 30
8.1.1 Subintervals of the naturals . . . 31
8.2 Processing two lists . . . 31
8.2.1 Case 1: one list goes along for the ride . . . 32
8.2.2 Case 2: processing in lockstep . . . 32
8.2.3 Case 3: processing at different rates . . . 32
8.2.4 Consuming a list and a number . . . 33
8.3 Summary . . . 33

9 Patterns of Recursion and Dictionaries 34
9.1 A catalogue of patterns . . . 34
9.2 Dictionaries . . . 34
9.3 Association lists . . . 35
9.3.1 Lookup . . . 35
9.3.2 Add or update . . . 35
9.3.3 Remove . . . 36
9.3.4 Efficiency . . . 36
9.4 Application: word frequencies . . . 36
9.5 Pairs: lists or structures? . . . 37
9.6 Summary . . . 37

10 Trees 38
10.1 Terminology . . . 38
10.2 Binary trees . . . 38
10.3 Binary search trees . . . 39
10.3.1 Search . . . 39
10.3.2 Insertion . . . 40
10.3.3 Traversal . . . 40
10.4 Evaluation (expression) trees . . . 40
10.5 General trees and mutual recursion . . . 41
10.5.1 Condensed trace . . . 42
10.6 Nested lists and leaf-labelled trees . . . 42
10.7 Summary . . . 42

11 Local Definitions and Lexical Scope 43
11.1 The `local` form . . . 43
11.2 Uses of local . . . 43
11.2.1 1. Clarity: naming intermediate values . . . 43
11.2.2 2. Efficiency: avoiding repeated computation . . . 43
11.2.3 3. Encapsulation: hiding helper functions . . . 44
11.2.4 4. Using enclosing parameters . . . 44
11.3 Semantics of local . . . 44
11.4 Lexical scope . . . 45
11.5 Style . . . 45
11.6 Summary . . . 46

12 Functional Abstraction **47**
12.1 Functions as arguments . . . 47
12.2 lambda: anonymous functions . . . 47
12.3 Functions as results . . . 48
12.4 The built-in abstract list functions . . . 48
12.4.1 map . . . 48
12.4.2 foldr . . . 49
12.4.3 foldl . . . 49
12.4.4 build-list . . . 49
12.4.5 Combining abstractions . . . 50
12.5 Abstraction with local and lambda . . . 50
12.6 Functional abstraction on other data . . . 50
12.7 When to use what . . . 50
12.8 Contracts for higher-order functions . . . 51
12.9 Summary . . . 51

13 Generative Recursion **52**
13.1 Euclid's algorithm . . . 52
13.1.1 A function that may not terminate . . . 53
13.2 Quicksort . . . 53
13.3 Mergesort . . . 54
13.4 Other examples of generative recursion . . . 54
13.4.1 Binary search for a root (bisection) . . . 54
13.4.2 Processing strings in chunks . . . 55
13.5 Design recipe for generative recursion . . . 55
13.6 Summary . . . 55

14 Graphs **56**
14.1 Definitions . . . 56
14.2 Representation: adjacency lists . . . 56
14.3 Finding paths . . . 57
14.3.1 The problem with cycles . . . 58
14.3.2 Fix: an accumulator of visited nodes . . . 58
14.4 Other graph problems . . . 59
14.5 Summary . . . 59

15 Efficiency, Computing History, and What's Next **60**
15.1 Measuring efficiency . . . 60
15.1.1 Typical running times in CS 135 . . . 60
15.1.2 Analysing recursive functions . . . 60
15.2 A short history of computing . . . 61
15.2.1 Before electronic computers . . . 61
15.2.2 Foundations: what can be computed? . . . 61
15.2.3 Electronic computers and programming languages . . . 62
15.3 What CS 135 did not cover . . . 62
15.4 What's next . . . 62
15.5 Summary . . . 63

Chapter 1

Introduction to CS 135 and Racket

1.1 What this course is about

CS 135 is the first course in the CS major sequence at Waterloo. Rather than starting with an industrial language such as C++ or Java, it uses **Racket**, a dialect of Scheme/Lisp, through a series of restricted *teaching languages* in the DrRacket environment:

- **Beginning Student** (BSL) — functions, constants, `cond`, structures, lists.
- **Beginning Student with List Abbreviations** — adds `list` and quote notation.
- **Intermediate Student** — adds `local`.
- **Intermediate Student with lambda** (ISL+) — adds anonymous functions and functions as first-class values.

The emphasis is on *design*: how to go systematically from a problem statement to a correct, readable, tested program. The tool for this is the **design recipe** (Chapter 2). The programming style is **functional**: there is no mutation and no assignment statement. A program is a collection of function definitions, and running it means *evaluating an expression*.

> Why a functional language first? Because without mutation, the meaning of a program can be explained entirely by algebraic substitution, the same way you simplify $f(3)$ in high-school algebra. That makes reasoning about correctness much easier, and everyone starts on equal footing regardless of prior programming experience.

1.2 From mathematical functions to Racket

In mathematics we write $f(x) = x^2$ and $g(x, y) = x + y$, and evaluate $g(f(2), f(3)) = g(4, 9) = 13$. Racket uses **prefix notation** with parentheses around every application:

```
(define (f x) (* x x))
(define (g x y) (+ x y))
(g (f 2) (f 3))   ; => 13
```

Points to note:

- The function (or operator) always comes first: `(+ 1 2)`, not `1 + 2`.

- Every parenthesis matters. `(f 2)` is an application; `((f 2))` tries to apply the number 4 as a function and is an error.
- There is no operator precedence to memorize: `(* 2 (+ 3 4))` is unambiguous.
- Many built-ins are variadic: `(+ 1 2 3 4)` is 10.

1.2.1 Numbers

Racket numbers are *exact* whenever possible. Integers are unbounded and rational arithmetic is exact: `(/ 1 3)` is the rational 1/3, not 0.333 . . .. Only operations like `sqrt` on non-perfect squares produce *inexact* numbers, displayed with a `#i` prefix (e.g. `#i1.414...`). Inexact numbers cannot be compared reliably with `check-expect`; use `check-within` instead.

Useful built-ins: `quotient`, `remainder`, `modulo`, `expt`, `sqrt`, `abs`, `max`, `min`, `floor`, `ceiling`, `exact->inexact`, `even?`, `odd?`, `zero?`.

1.3 Expressions, constants and functions

A Racket program is a sequence of **definitions** and **expressions**.

Definition 1.1 (Constant definition)**.** `(define name expr)` evaluates `expr` to a value and binds `name` to it.

Definition 1.2 (Function definition)**.** `(define (fname x1 ... xn) body)` creates a function with **parameters** `x1 ... xn`. When applied to **arguments**, the body is evaluated with each parameter replaced by the corresponding argument value.

```
(define tax-rate 13/100)
(define (price-with-tax p)
  (+ p (* p tax-rate)))
(price-with-tax 100)   ; => 113
```

Constants make programs easier to read and change: if the tax rate changes, it is updated in one place. Avoid "magic numbers" in function bodies.

1.4 Syntax and semantics

Every language has a **syntax** (which programs are well-formed) and a **semantics** (what a well-formed program means). For natural languages: "Trombones fly hungrily" is syntactically fine but semantically nonsense.

1.4.1 Syntax

Racket syntax is very small. Ignoring later additions, an expression is one of:

- a value (number, boolean, string, symbol, . . .);
- an identifier (name of a constant or parameter);
- an application `(f e1 ... en)` where `f` is a function name and each `ei` is an expression;

- a special form such as `(cond [q1 a1] ...)`, `(and ...)`, `(or ...)`.

Errors such as unbalanced parentheses or `(define 5 x)` are *syntax errors*: DrRacket refuses to run the program at all.

1.4.2 Semantics: the substitution model

CS 135 gives Racket a precise semantics as a set of **rewrite (reduction) rules**. A program is evaluated by repeatedly finding the leftmost innermost subexpression that can be simplified, rewriting it, and continuing until only values remain. Each rewrite is one *step*, written ⇒.

Substitution rules (Beginning Student)

1. **Built-in application**: `(f v1 ... vn)` ⇒ `v`, where `f` is built in, the `vi` are values, and `v` is the result of applying `f`.
2. **User-defined application**: if `(define (f x1 ... xn) exp)` occurs earlier, then `(f v1 ... vn)` ⇒ `exp'`, where `exp'` is `exp` with every `xi` replaced by `vi`.
3. **Constant**: if `(define id val)` occurs earlier, then `id` ⇒ `val`.
4. Arguments are evaluated left to right, and *all arguments must be values* before rules 1 or 2 apply.

Constant definitions are themselves evaluated: `(define x (+ 1 2))` becomes `(define x 3)` before later expressions can use `x`.

Example 1.3 (Stepper trace)**.** Using the definitions of `f` and `g` above:

```
(g (f 2) (f 3))
=> (g (* 2 2) (f 3))      ; rule 2 on (f 2)
=> (g 4 (f 3))            ; rule 1
=> (g 4 (* 3 3))          ; rule 2 on (f 3)
=> (g 4 9)                ; rule 1
=> (+ 4 9)                ; rule 2 on g
=> 13                     ; rule 1
```

Note that `(f 2)` is rewritten to `(* 2 2)` in one step (the body with substitution), not directly to `4`. Exam questions frequently ask for the *exact* sequence of steps, so be precise about this. DrRacket's **Stepper** tool shows exactly these steps and is the best way to check your traces.

Example 1.4 (Trace with a constant)**.**

```
(define tax-rate 13/100)
(price-with-tax 200)
=> (+ 200 (* 200 tax-rate))
=> (+ 200 (* 200 13/100))
=> (+ 200 26)
=> 226
```

1.4.3 Run-time errors

A syntactically valid program can still fail during evaluation, e.g. `(/ 1 0)` or `(f 1 2)` when `f` takes one argument. In the substitution model this shows up as an expression that is not a value but to which no rule applies; evaluation stops with an error.

1.5 Identifiers and scope (preview)

Parameter names are local to their function: the `x` in `f` has nothing to do with an `x` in `g`. Constants defined at the top level are visible everywhere after their definition. A parameter with the same name as a constant *shadows* the constant inside that function body. Chapter 11 makes this precise.

Identifiers in Racket may contain almost any character except spaces, parentheses and a few others, so `price-with-tax`, `empty?`, `->string` and `x*` are all legal names. Conventionally, names are lowercase words separated by hyphens, and predicates end in `?`.

1.6 Programs in DrRacket

- The **definitions window** holds the program; **Run** evaluates it top to bottom and shows results of top-level expressions in the **interactions window**.
- Tests (`check-expect`) are collected and run after the whole program, so a test may appear before the function it tests.
- Choose the correct language level from *Language* → *Choose Language*. Using a feature not in the current level is a syntax error; this is intentional, to catch typical beginner mistakes with clear messages.

1.7 Common pitfalls

- Extra or missing parentheses: `(define (f x) (x))` applies `x` as a function.
- Infix habits: `(2 + 3)` is an error.
- Forgetting that all arguments are evaluated first, so a trace that rewrites the outer function too early is wrong.
- Comparing inexact numbers with `check-expect`.

1.8 Summary

- CS 135 teaches program design using Racket teaching languages, without mutation.
- Racket uses fully parenthesized prefix notation; a program is definitions plus expressions.
- Semantics is given by the substitution model: evaluate arguments left to right to values, then substitute into the function body; constants are replaced by their values.
- Know how to produce a precise step-by-step trace; the Stepper is the reference.

Chapter 2

The Design Recipe

2.1 Why a recipe?

Programs are read far more often than they are written. The design recipe is a fixed process that produces, for every function, documentation and tests in addition to code. It also gives you somewhere to start when you are stuck: each step produces something concrete that guides the next step. In CS 135, marks are given for every component, not only for working code.

The design recipe (for each function)

1. **Purpose**: a short description of what the function computes, mentioning every parameter by name.
2. **Contract**: the types of the consumed values and the produced value.
3. **Examples**: a few illustrative cases written as `check-expect`s, written *before* the body.
4. **Definition**: the header and body (often derived from a *template*).
5. **Tests**: a thorough set of `check-expect`s covering edge cases and every branch.

The order of *writing* is roughly: contract and purpose first (to understand the problem), then examples (to understand the answer), then the header and body, then tests. The standard order *in the file* is:

```
;; (sum-of-squares a b) produces the sum of the squares of a and b.
;; Examples:
(check-expect (sum-of-squares 3 4) 25)
(check-expect (sum-of-squares 0 0) 0)

;; sum-of-squares: Num Num -> Num
(define (sum-of-squares a b)
  (+ (* a a) (* b b)))

;; Tests:
(check-expect (sum-of-squares -2 5) 29)
(check-expect (sum-of-squares 1/2 1/2) 1/2)
```

2.2 The components in detail

2.2.1 Purpose

The purpose statement starts with the function applied to its parameter names, for example `;; (area-of-ring outer inner) produces ...`. It says *what* is produced, not *how*. Avoid restating the code ("adds a times a to b times b").

2.2.2 Contract

The contract has the form `name: Type1 Type2 ... -> TypeOut`. The types used in CS 135 include:

Type	Meaning
`Num`	any number
`Int`	integer
`Nat`	natural number $0, 1, 2, \ldots$
`Bool`	`true` or `false`
`Sym`	symbol, e.g. `'red`
`Str`	string, e.g. `"hello"`
`Char`	character, e.g. `#\a`
`Any`	any value
`(listof X)`	list whose elements have type `X`
`(anyof X Y)`	a value that is of type X or of type Y
`(X -> Y)`	a function consuming X and producing Y

When the type alone is not specific enough, add a **requires** clause:

```
;; safe-div: Num Num -> Num
;;   requires: d is not 0
(define (safe-div n d) (/ n d))
```

The function does not have to check its requirements; calling it with values that violate the contract is the caller's error.

2.2.3 Examples

Examples are written before the body, using values worked out by hand. They clarify the problem and become the first tests. A useful habit: if you cannot write an example, you do not yet understand the problem.

2.2.4 Tests

- `(check-expect actual expected)`: passes if the two values are equal.
- `(check-within actual expected tolerance)`: for inexact numbers.
- `(check-error expr "message")`: for expressions expected to raise an error.

Good tests are **white-box** as well as **black-box**: exercise every `cond` branch, boundary values (0, empty list, equal arguments), and typical values. DrRacket highlights code never executed by any test in black; aim for full coverage.

2.3 Data definitions and templates

For functions that consume structured data (structures, lists, trees), the recipe gains two steps that are done *once per data type*, not once per function:

1. A **data definition**: a comment describing the set of values of the new type, often recursively.
2. A **template**: a skeleton function that takes apart a value of that type in the way the data definition suggests.

Every function consuming that type begins as a copy of the template. Later chapters develop templates for structures, lists, natural numbers, and trees.

2.4 A complete example

Example 2.1. Write a function that computes the area of a ring with given outer and inner radii.

```
;; (area-of-ring outer inner) produces the area of a ring whose
;;   outer radius is outer and inner (hole) radius is inner.
;; Examples:
(check-within (area-of-ring 2 1) (* pi 3) 0.0001)
(check-within (area-of-ring 1 1) 0 0.0001)

;; area-of-ring: Num Num -> Num
;;   requires: outer >= inner >= 0
(define (area-of-ring outer inner)
  (- (area-of-disk outer) (area-of-disk inner)))

;; (area-of-disk r) produces the area of a disk of radius r.
;; Example:
(check-within (area-of-disk 1) pi 0.0001)
;; area-of-disk: Num -> Num
;;   requires: r >= 0
(define (area-of-disk r) (* pi r r))

;; Tests:
(check-within (area-of-ring 10 0) (* pi 100) 0.0001)
(check-within (area-of-ring 3 2) (* pi 5) 0.0001)
```

Note the **helper function** `area-of-disk`. Helpers get their own full design recipe (purpose, contract, examples, tests), though in some terms only purpose and contract were required for helpers. Introduce a helper whenever you notice repeated code, or when a subtask has a natural name.

2.5 Style conventions

- Use meaningful names; hyphenate multi-word names (`area-of-ring`).
- Indent consistently; DrRacket's *Reindent All* (Ctrl-I) does it for you.
- Keep lines under about 80 characters.

- Define constants for repeated literal values.
- Put a header comment with name, ID and assignment at the top of each file.

2.6 Common pitfalls

- Writing examples after the code by copying the output — this tests nothing.
- Purpose statements that do not mention the parameters.
- Using `check-expect` on inexact values.
- Contracts that are too vague (`Any` when `Nat` is meant) or omit requirements.

2.7 Summary

- Purpose, contract, examples, definition, tests — for every function including helpers.
- Contracts use a fixed vocabulary of types plus `requires` clauses.
- Data definitions and templates are written once per type and reused.
- Tests should cover every branch and boundary case.

Chapter 3

Simple Data

This chapter introduces the atomic (non-compound) data types beyond numbers — booleans, symbols, strings, and characters — together with the conditional expression `cond`, which lets a function choose among cases.

3.1 Booleans and predicates

The two Boolean values are `true` and `false` (also written `#true`/`#t` and `#false`/`#f`). A **predicate** is a function that produces a Boolean; by convention its name ends in `?`.

- Comparisons: `=`, `<`, `<=`, `>`, `>=` on numbers.
- Type predicates: `number?`, `integer?`, `boolean?`, `symbol?`, `string?`, `char?`.
- Equality: `equal?` works on any two values; `symbol=?`, `string=?`, `char=?` are type-specific.

3.1.1 `and`, `or`, `not`

`not` is an ordinary function. `and` and `or` are **special forms**: they evaluate their arguments left to right and stop as soon as the answer is known (**short-circuit evaluation**).

Substitution rules for and/or

- `(and false ...)` ⇒ `false`
- `(and true e2 ...)` ⇒ `(and e2 ...)`
- `(and)` ⇒ `true`
- `(or true ...)` ⇒ `true`
- `(or false e2 ...)` ⇒ `(or e2 ...)`
- `(or)` ⇒ `false`

Short-circuiting lets us guard a dangerous test:

```
(and (not (= x 0)) (> (/ 1 x) 2))   ; never divides by zero
```

3.2 Conditional expressions

```
(cond
  [question1 answer1]
  [question2 answer2]
  ...
  [else answerk])
```

Each `[question answer]` pair is a **clause**. Questions are tried in order; the answer of the first question that evaluates to `true` is the result.

Substitution rules for cond

- `(cond [false e] ...)` ⇒ `(cond ...)`
- `(cond [true e] ...)` ⇒ `e`
- `(cond [else e])` ⇒ `e`

Only the *first* question is evaluated at each step; answers of unchosen clauses are never evaluated. If no question is true and there is no `else`, it is a run-time error.

Example 3.1 (Grading).

```
;; (letter-grade mark) produces the letter grade for a percentage mark.
;; Examples:
(check-expect (letter-grade 95) 'A)
(check-expect (letter-grade 42) 'F)

;; letter-grade: Num -> Sym
;;   requires: 0 <= mark <= 100
(define (letter-grade mark)
  (cond [(>= mark 80) 'A]
        [(>= mark 70) 'B]
        [(>= mark 60) 'C]
        [(>= mark 50) 'D]
        [else 'F]))

;; Tests (boundaries of every interval):
(check-expect (letter-grade 80) 'A)
(check-expect (letter-grade 79) 'B)
(check-expect (letter-grade 70) 'B)
(check-expect (letter-grade 60) 'C)
(check-expect (letter-grade 50) 'D)
(check-expect (letter-grade 49.9) 'F)
```

Because clauses are tried in order, the second clause need not say `(and (< mark 80) (>= mark 70))`. Take advantage of this, but be careful when reordering.

Example 3.2 (Trace of cond).

```
(letter-grade 75)
=> (cond [(>= 75 80) 'A] [(>= 75 70) 'B] [(>= 75 60) 'C] [(>= 75 50) 'D] [else 'F])
=> (cond [false 'A] [(>= 75 70) 'B] [(>= 75 60) 'C] [(>= 75 50) 'D] [else 'F])
=> (cond [(>= 75 70) 'B] [(>= 75 60) 'C] [(>= 75 50) 'D] [else 'F])
=> (cond [true 'B] [(>= 75 60) 'C] [(>= 75 50) 'D] [else 'F])
=> 'B
```

3.2.1 Simplifying conditionals

- `(cond [test true] [else false])` is just `test`.
- Nested `cond`s whose clauses can be merged with `and` often read better flattened.
- A function that produces a Boolean rarely needs `cond` at all; combine `and`/`or`/`not`.

3.2.2 Testing conditionals

At least one test per clause, plus tests at each boundary between intervals. For a `cond` with k clauses expect at least k tests; boundary values are where off-by-one bugs live.

3.3 Symbols

A **symbol** is written with a leading quote: `'blue`, `'CS135`. Symbols are atomic — they cannot be taken apart — and the only useful operation is comparison with `symbol=?` (or `equal?`). They are ideal for representing a small fixed set of options, like an enumeration:

```
;; A TrafficLight is (anyof 'red 'yellow 'green)

;; (next-light c) produces the light shown after c.
;; next-light: TrafficLight -> TrafficLight
(check-expect (next-light 'red) 'green)
(define (next-light c)
  (cond [(symbol=? c 'red) 'green]
        [(symbol=? c 'green) 'yellow]
        [(symbol=? c 'yellow) 'red]))
```

3.4 Strings

A **string** is a sequence of characters in double quotes: `"CS 135"`. Unlike symbols, strings have structure and many operations:

Function	Example
`string-length`	`(string-length "abc")` ⇒ 3
`string-append`	`(string-append "ab" "cd")` ⇒ `"abcd"`
`substring`	`(substring "hello" 1 3)` ⇒ `"el"`
`string=?`, `string<?`	lexicographic comparison
`number->string`	`(number->string 42)` ⇒ `"42"`
`string->list`	`(string->list "ab")` ⇒ `(list #\a #\b)`
`list->string`	inverse of the above

Note that `substring` uses a half-open range: start index included, end index excluded, with indices starting at 0.

Symbol or string? Use a symbol when the value is a label you only compare (colours, suits, states). Use a string when the value is text that might be examined, combined, or displayed.

3.5 Characters

A **character** is written `#\a`, `#\Z`, `#\space`, `#\newline`. Useful functions:

- comparison: `char=?`, `char<?`;
- case conversion: `char-upcase`, `char-downcase`;
- classification: `char-alphabetic?`, `char-numeric?`, `char-whitespace?`;
- conversion to and from character codes: `char->integer`, `integer->char`.

Characters become important once strings are converted to lists of characters (Chapter 5).

3.6 Mixed data and type predicates

Sometimes a function accepts values of several types. The contract uses `anyof`, and the body uses type predicates to distinguish the cases:

```
;; (describe v) produces a string describing v.
;; describe: (anyof Num Sym Str) -> Str
(check-expect (describe 5) "number")
(check-expect (describe 'x) "symbol")
(check-expect (describe "hi") "string")
(define (describe v)
  (cond [(number? v) "number"]
        [(symbol? v) "symbol"]
        [else "string"]))
```

3.7 Summary

- Booleans, symbols, strings and characters are the atomic types besides numbers.
- `and`/`or` short-circuit; `cond` evaluates questions in order and only the chosen answer.
- Know the exact substitution rules for `cond`, `and`, `or`.
- Test every clause and every boundary.
- Symbols for labels, strings for text; `anyof` plus type predicates for mixed data.

Chapter 4

Structures

Atomic data cannot represent compound information such as a point (x, y), a student record, or a date. **Structures** bundle a fixed number of values into one value.

4.1 Defining a structure

```
(define-struct posn (x y))
```

(In fact `posn` is built into the teaching languages, but this is how it would be defined.) A single `define-struct` creates several functions:

- **constructor** `make-posn`: `Num Num -> Posn`;
- **selectors** `posn-x` and `posn-y`: `Posn -> Num`;
- **type predicate** `posn?`: `Any -> Bool`.

```
(define p (make-posn 3 4))
(posn-x p)        ; => 3
(posn-y p)        ; => 4
(posn? p)         ; => true
(posn? 7)         ; => false
```

A value such as `(make-posn 3 4)` is itself a *value*: it cannot be simplified further. Structures are immutable in CS 135; to "change" a field you construct a new structure.

4.1.1 Substitution rules for structures

Structure rules

For `(define-struct sname (f1 ... fn))`:

- `(sname-fi (make-sname v1 ... vn))` ⇒ `vi`
- `(sname? (make-sname v1 ... vn))` ⇒ `true`
- `(sname? v)` ⇒ `false` for any other value `v`

4.2 Data definitions and structure definitions

Every `define-struct` is accompanied by a comment called the **structure definition** (field names and types) and a **data definition** giving the type a capitalized name:

```
(define-struct student (name id avg))
;; A Student is a (make-student Str Nat Num)
;;   requires: 0 <= avg <= 100
```

4.3 Templates for structures

A template for a function consuming a structure selects each field, since a function consuming a Student will usually need some subset of its fields:

```
;; student-template: Student -> Any
(define (student-template s)
  (... (student-name s) ...
       (student-id s) ...
       (student-avg s) ...))
```

Example 4.1 (Distance between points)**.**

```
;; (distance p1 p2) produces the Euclidean distance between p1 and p2.
;; Examples:
(check-expect (distance (make-posn 0 0) (make-posn 3 4)) 5)
(check-expect (distance (make-posn 1 1) (make-posn 1 1)) 0)

;; distance: Posn Posn -> Num
(define (distance p1 p2)
  (sqrt (+ (sqr (- (posn-x p1) (posn-x p2)))
           (sqr (- (posn-y p1) (posn-y p2))))))

;; Tests:
(check-within (distance (make-posn 0 0) (make-posn 1 1)) (sqrt 2) 0.0001)
```

Example 4.2 (Producing a new structure)**.**

```
;; (curve-avg s bonus) produces a copy of s with bonus added to the
;;   average, capped at 100.
;; curve-avg: Student Num -> Student
(check-expect (curve-avg (make-student "Ada" 1 70) 5)
              (make-student "Ada" 1 75))
(check-expect (curve-avg (make-student "Bo" 2 98) 5)
              (make-student "Bo" 2 100))
(define (curve-avg s bonus)
  (make-student (student-name s)
                (student-id s)
                (min 100 (+ (student-avg s) bonus))))
```

Trace.

```
(curve-avg (make-student "Ada" 1 70) 5)
=> (make-student (student-name (make-student "Ada" 1 70))
                 (student-id (make-student "Ada" 1 70))
                 (min 100 (+ (student-avg (make-student "Ada" 1 70)) 5)))
```

```
=> (make-student "Ada"
                 (student-id (make-student "Ada" 1 70))
                 (min 100 (+ (student-avg (make-student "Ada" 1 70)) 5)))
=> (make-student "Ada" 1 (min 100 (+ (student-avg (make-student "Ada" 1 70)) 5)))
=> (make-student "Ada" 1 (min 100 (+ 70 5)))
=> (make-student "Ada" 1 (min 100 75))
=> (make-student "Ada" 1 75)
```

4.4 Nested structures

Fields can themselves be structures. The template then calls the template of the inner type (or a helper) on that field.

```
(define-struct circle (centre radius))
;; A Circle is a (make-circle Posn Num)
;;   requires: radius > 0

;; circle-template: Circle -> Any
(define (circle-template c)
  (... (posn-template (circle-centre c)) ...
       (circle-radius c) ...))

;; (inside? c p) produces true if p lies inside or on circle c.
;; inside?: Circle Posn -> Bool
(check-expect (inside? (make-circle (make-posn 0 0) 5) (make-posn 3 4)) true)
(check-expect (inside? (make-circle (make-posn 0 0) 5) (make-posn 4 4)) false)
(define (inside? c p)
  (<= (distance (circle-centre c) p) (circle-radius c)))
```

4.5 Mixed data with structures

A type may be one of several structure types. The data definition uses `anyof` and the template uses the type predicates in a `cond`:

```
(define-struct rect (corner width height))
;; A Rect is a (make-rect Posn Num Num)
;; A Shape is (anyof Circle Rect)

;; shape-template: Shape -> Any
(define (shape-template sh)
  (cond [(circle? sh) (... (circle-centre sh) ... (circle-radius sh) ...)]
        [(rect? sh) (... (rect-corner sh) ... (rect-width sh) ...
                         (rect-height sh) ...)]))

;; (area sh) produces the area of shape sh.
;; area: Shape -> Num
(check-expect (area (make-rect (make-posn 0 0) 2 3)) 6)
(define (area sh)
  (cond [(circle? sh) (* pi (sqr (circle-radius sh)))]
        [(rect? sh) (* (rect-width sh) (rect-height sh))]))
```

Racket is *dynamically typed*: nothing stops you from writing `(make-posn "a" 'b)`. Contracts and data definitions are comments that document intended use. Selectors do check their argument's structure type, so `(posn-x 5)` is a run-time error.

4.6 Common pitfalls

- Forgetting the data definition comment or field types.
- Writing `(x p)` instead of `(posn-x p)`.
- Trying to mutate a field; construct a new structure instead.
- Comparing structures with `=`; use `equal?` (and `check-expect` compares structurally).

4.7 Summary

- `define-struct` creates a constructor, selectors and a predicate.
- Every structure has a structure definition and a data definition comment.
- The structure template selects every field; nested structures lead to nested template calls.
- Mixed data uses `anyof` and type predicates in a `cond`.

Chapter 5

Lists

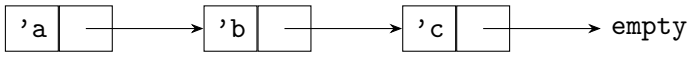
Structures hold a *fixed* number of values. To represent an arbitrary number of values — a class list, the words in a sentence, a sequence of measurements — we need a type whose values can be any size. The list is the fundamental such type in Racket.

5.1 Constructing lists

A list is either empty, or a first element followed by the rest of the list.

- `empty` (also written `'()`) is the empty list.
- `(cons v lst)` produces a new list whose first element is `v` and whose rest is the list `lst`.

```
empty                                   ; the empty list
(cons 'a empty)                         ; one element
(cons 'a (cons 'b (cons 'c empty)))     ; three elements: a, b, c
```



5.2 Deconstructing lists

- `(first lst)` — the first element; error on `empty`.
- `(rest lst)` — the list without its first element; error on `empty`.
- `(empty? v)` — true iff `v` is the empty list.
- `(cons? v)` — true iff `v` is a non-empty list.

List substitution rules

- `(first (cons a b))` ⇒ `a`
- `(rest (cons a b))` ⇒ `b`
- `(empty? empty)` ⇒ `true`; `(empty? v)` ⇒ `false` for any other value
- `(cons? (cons a b))` ⇒ `true`; `(cons? v)` ⇒ `false` otherwise

A value `(cons v1 (cons v2 empty))` is fully evaluated: `cons` is a constructor.

5.3 Data definition for lists

The set of lists is defined **recursively** (self-referentially):

```
;; A (listof Num) is one of:
;; * empty
;; * (cons Num (listof Num))
```

The definition is not circular because the base case `empty` is not self-referential, and every self-reference refers to a smaller list. We use `(listof X)` for any element type `X`, and sometimes `(ne-listof X)` for non-empty lists:

```
;; A (ne-listof X) is one of:
;; * (cons X empty)
;; * (cons X (ne-listof X))
```

5.4 The list template

The template follows the data definition: one `cond` clause per case, selectors in the `cons` case, and — because the rest is itself a list — a **recursive call** on the rest.

```
;; listof-X-template: (listof X) -> Any
(define (listof-X-template lst)
  (cond [(empty? lst) ...]
        [(cons? lst) (... (first lst) ...
                          (listof-X-template (rest lst)) ...)]))
```

Key idea

Assume the recursive call already produces the correct answer for `(rest lst)`. Your job is only to say how to combine `(first lst)` with that answer. This is the programming analogue of the induction step in a proof by induction.

5.5 First recursive functions

Example 5.1 (Counting elements)**.**

```
;; (count-items lst) produces the number of elements in lst.
;; Examples:
```

```
(check-expect (count-items empty) 0)
(check-expect (count-items (cons 'a (cons 'b empty))) 2)

;; count-items: (listof Any) -> Nat
(define (count-items lst)
  (cond [(empty? lst) 0]
        [else (+ 1 (count-items (rest lst)))]))
```

(This is the built-in `length`.)

Example 5.2 (Sum).

```
;; (sum lst) produces the sum of the numbers in lst.
;; Examples:
(check-expect (sum empty) 0)
(check-expect (sum (cons 2 (cons 3 empty))) 5)

;; sum: (listof Num) -> Num
(define (sum lst)
  (cond [(empty? lst) 0]
        [else (+ (first lst) (sum (rest lst)))]))
```

Condensed trace. Full traces of recursive functions are long; a *condensed trace* shows only the key steps:

```
(sum (cons 2 (cons 3 empty)))
=> (+ 2 (sum (cons 3 empty)))
=> (+ 2 (+ 3 (sum empty)))
=> (+ 2 (+ 3 0))
=> (+ 2 3)
=> 5
```

For comparison, the first few full steps are:

```
(sum (cons 2 (cons 3 empty)))
=> (cond [(empty? (cons 2 (cons 3 empty))) 0]
         [else (+ (first (cons 2 (cons 3 empty)))
                  (sum (rest (cons 2 (cons 3 empty)))))])
=> (cond [false 0]
         [else (+ (first (cons 2 (cons 3 empty)))
                  (sum (rest (cons 2 (cons 3 empty)))))])
=> (cond [else (+ (first (cons 2 (cons 3 empty)))
                  (sum (rest (cons 2 (cons 3 empty)))))])
=> (+ (first (cons 2 (cons 3 empty))) (sum (rest (cons 2 (cons 3 empty)))))
=> (+ 2 (sum (rest (cons 2 (cons 3 empty)))))
=> (+ 2 (sum (cons 3 empty)))
=> ...
```

5.6 List abbreviations

In *Beginning Student with List Abbreviations*:

- `(list 1 2 3)` abbreviates `(cons 1 (cons 2 (cons 3 empty)))`.

- `'(a b c)` abbreviates `(list 'a 'b 'c)`; quoting distributes over the elements.
- `second`, `third`, ..., `eighth` select later elements.

`(list 1 2)` and `(cons 1 2)` are different: the latter is not a list at all in the teaching languages (it is an error, since the second argument must be a list). `(cons (list 1 2) (list 3))` is the two-element list `(list (list 1 2) 3)`.

5.7 Lists of structures and strings

Lists can hold any values. A list of structures uses both templates together:

```
;; (class-average los) produces the average avg of the students in los.
;; class-average: (listof Student) -> Num
;;   requires: los is non-empty
(define (class-average los)
  (/ (total-avg los) (length los)))

;; (total-avg los) produces the sum of the averages in los.
;; total-avg: (listof Student) -> Num
(define (total-avg los)
  (cond [(empty? los) 0]
        [else (+ (student-avg (first los)) (total-avg (rest los)))]))
```

Strings are processed by converting to a list of characters:

```
;; (count-vowels s) produces the number of vowels in s.
;; count-vowels: Str -> Nat
(check-expect (count-vowels "banana") 3)
(define (count-vowels s) (count-vowels/list (string->list s)))

;; count-vowels/list: (listof Char) -> Nat
(define (count-vowels/list loc)
  (cond [(empty? loc) 0]
        [(vowel? (first loc)) (+ 1 (count-vowels/list (rest loc)))]
        [else (count-vowels/list (rest loc))]))

;; vowel?: Char -> Bool
(define (vowel? c) (member? (char-downcase c) '(#\a #\e #\i #\o #\u)))
```

5.8 Common pitfalls

- Taking `first` or `rest` of `empty`.
- Forgetting the base case, or getting its value wrong (the sum of nothing is 0, the product of nothing is 1).
- Confusing `cons` (add one element at the front) with `append` (join two lists).
- Making the recursive call on the whole list instead of the rest (infinite recursion).

5.9 Summary

- A list is `empty` or `(cons first rest)`; the data definition is recursive.
- The list template has an `empty` case and a `cons` case with a recursive call on the rest.
- Trust the recursion: combine the first element with the answer for the rest.
- `list` and quote are abbreviations for nested `cons`.

Chapter 6

Working with Recursion on Lists

Having seen the list template, we now practise **structural recursion**: recursion whose shape follows exactly the shape of the data definition.

Definition 6.1 (Pure structural recursion)**.** A function uses **pure structural recursion** if every recursive call is on one of the self-referential parts of the data (for a list, `(rest lst)`) and all other parameters are passed unchanged.

6.1 Functions producing Booleans

```
;; (contains? item lst) produces true if item is in lst.
;; Examples:
(check-expect (contains? 'b '(a b c)) true)
(check-expect (contains? 'z empty) false)

;; contains?: Any (listof Any) -> Bool
(define (contains? item lst)
  (cond [(empty? lst) false]
        [else (or (equal? item (first lst))
                  (contains? item (rest lst)))]))
```

Because `or` short-circuits, the recursion stops as soon as the item is found. (Built-in: `member?`.)

6.2 Functions producing lists

6.2.1 Transforming each element (a "map" pattern)

```
;; (double-all lon) produces a list with each number in lon doubled.
;; Examples:
(check-expect (double-all empty) empty)
(check-expect (double-all '(1 2 3)) '(2 4 6))

;; double-all: (listof Num) -> (listof Num)
(define (double-all lon)
  (cond [(empty? lon) empty]
        [else (cons (* 2 (first lon)) (double-all (rest lon)))]))
```

6.2.2 Selecting some elements (a "filter" pattern)

```
;; (keep-positive lon) produces the positive numbers of lon, in order.
;; Examples:
(check-expect (keep-positive '(3 -1 0 5)) '(3 5))

;; keep-positive: (listof Num) -> (listof Num)
(define (keep-positive lon)
  (cond [(empty? lon) empty]
        [(positive? (first lon)) (cons (first lon) (keep-positive (rest lon)))]
        [else (keep-positive (rest lon))]))
```

These patterns recur so often that Chapter 12 abstracts them into the built-in higher-order functions `map` and `filter`.

6.2.3 Appending

```
;; (my-append l1 l2) produces the elements of l1 followed by those of l2.
;; my-append: (listof Any) (listof Any) -> (listof Any)
(check-expect (my-append '(1 2) '(3 4)) '(1 2 3 4))
(define (my-append l1 l2)
  (cond [(empty? l1) l2]
        [else (cons (first l1) (my-append (rest l1) l2))]))
```

Only `l1` is recursed on; `l2` goes along unchanged. This is still pure structural recursion on `l1`.

6.3 Non-empty lists

Some functions make no sense on `empty`, such as the maximum:

```
;; (list-max lon) produces the largest number in lon.
;; list-max: (ne-listof Num) -> Num
(check-expect (list-max '(3 9 2)) 9)
(check-expect (list-max '(4)) 4)
(define (list-max lon)
  (cond [(empty? (rest lon)) (first lon)]
        [else (max (first lon) (list-max (rest lon)))]))
```

The template for `(ne-listof X)` has base case `(empty? (rest lst))`.

> A version that computes `(list-max (rest lon))` *twice* (once in a comparison and once as the answer) takes exponential time: each call makes two recursive calls. Avoid repeating recursive calls; use `max`, a helper, or `local` (Chapter 11).

6.4 Insertion sort

Sorting is a classic application of structural recursion. If we can sort the rest of the list, we only need to *insert* the first element into the correct place.

```
;; (sort lon) produces lon sorted in non-decreasing order.
;; Examples:
```

```
(check-expect (sort empty) empty)
(check-expect (sort '(3 1 2)) '(1 2 3))

;; sort: (listof Num) -> (listof Num)
(define (sort lon)
  (cond [(empty? lon) empty]
        [else (insert (first lon) (sort (rest lon)))]))

;; (insert n slon) inserts n into the sorted list slon so that the
;;   result is sorted.
;; Examples:
(check-expect (insert 4 '(1 3 5)) '(1 3 4 5))
(check-expect (insert 0 empty) '(0))

;; insert: Num (listof Num) -> (listof Num)
;;   requires: slon is sorted in non-decreasing order
(define (insert n slon)
  (cond [(empty? slon) (cons n empty)]
        [(<= n (first slon)) (cons n slon)]
        [else (cons (first slon) (insert n (rest slon)))]))
```

Condensed trace.

```
(sort (list 3 1 2))
=> (insert 3 (sort (list 1 2)))
=> (insert 3 (insert 1 (sort (list 2))))
=> (insert 3 (insert 1 (insert 2 (sort empty))))
=> (insert 3 (insert 1 (insert 2 empty)))
=> (insert 3 (insert 1 (list 2)))
=> (insert 3 (list 1 2))
=> (cons 1 (insert 3 (list 2)))
=> (cons 1 (cons 2 (insert 3 empty)))
=> (list 1 2 3)
```

Insertion sort takes time proportional to n^2 in the worst case (a reverse-sorted list), since each of n insertions may traverse the whole sorted list. (The name `sort` clashes with a built-in in later levels; call it `insertion-sort` there.)

6.5 Lists of lists

A list element may itself be a list. With nested data, the template calls a helper (or another template) on `(first lst)`:

```
;; (sum-all lolon) produces the sum of all numbers in all lists of lolon.
;; sum-all: (listof (listof Num)) -> Num
(check-expect (sum-all '((1 2) () (3))) 6)
(define (sum-all lolon)
  (cond [(empty? lolon) 0]
        [else (+ (sum (first lolon)) (sum-all (rest lolon)))]))
```

6.6 Wrapper functions

A **wrapper** is a short non-recursive function that adapts its input and calls a recursive helper, e.g. `count-vowels` (Chapter 5) converts a string to a list of characters and calls the list function. The wrapper has the nice interface; the helper does the recursion.

6.7 Testing recursive functions

Test at least: the empty list; a one-element list; a longer list; lists where the interesting element is first, last, absent, or repeated; already sorted and reverse-sorted input for sorting functions.

6.8 Summary

- Structural recursion follows the data definition; one recursive call on `(rest lst)`.
- Common patterns: count/sum (combine), contains (search with `or`), map (transform each), filter (keep some).
- Non-empty lists have a base case at a one-element list.
- Insertion sort: sort the rest, then insert the first; $\mathcal{O}(n^2)$ worst case.
- Never repeat the same recursive call in one body.

Chapter 7

Types of Recursion

Structural recursion is the default, but some problems need more. CS 135 classifies recursive functions as follows:

- **Pure structural**: recursive calls only on the self-referential parts; other parameters unchanged.
- **Accumulative** (structural with an accumulator): recursion is structural on one parameter, but extra parameters *accumulate* information about what has been seen so far.
- **Mutual**: two or more functions call each other.
- **Generative**: recursive calls are on values *computed* from the input, not just its parts (Chapter 13).

7.1 Accumulative recursion

7.1.1 Motivation: reversing a list

A structural solution:

```
;; (my-reverse lst) produces lst in reverse order.
;; my-reverse: (listof Any) -> (listof Any)
(define (my-reverse lst)
  (cond [(empty? lst) empty]
        [else (append (my-reverse (rest lst)) (list (first lst)))]))
```

Each `append` walks the entire reversed rest, so the total work is $1 + 2 + \cdots + n = \Theta(n^2)$. With an accumulator we can do it in linear time:

```
;; (my-reverse lst) produces lst in reverse order.
;; Examples:
(check-expect (my-reverse '(1 2 3)) '(3 2 1))
(check-expect (my-reverse empty) empty)

;; my-reverse: (listof Any) -> (listof Any)
(define (my-reverse lst) (rev/acc lst empty))

;; (rev/acc lst acc) produces the elements of lst in reverse order
;;   followed by acc.
```

```
;; rev/acc: (listof Any) (listof Any) -> (listof Any)
(define (rev/acc lst acc)
  (cond [(empty? lst) acc]
        [else (rev/acc (rest lst) (cons (first lst) acc))]))
```

Trace.

```
(my-reverse (list 1 2 3))
=> (rev/acc (list 1 2 3) empty)
=> (rev/acc (list 2 3) (list 1))
=> (rev/acc (list 3) (list 2 1))
=> (rev/acc empty (list 3 2 1))
=> (list 3 2 1)
```

Accumulative recursion recipe

- A **wrapper** function calls the helper with the *initial* accumulator value.
- The helper's purpose explains what the accumulator *means* (the accumulator invariant).
- In the recursive call the structural parameter moves to the rest, and the accumulator is updated to reflect the element just processed.
- In the base case the answer is usually (computed from) the accumulator.

7.1.2 Template

```
;; acc-template: (listof X) Y -> Any
(define (acc-template lst acc)
  (cond [(empty? lst) (... acc ...)]
        [else (acc-template (rest lst)
                            (... (first lst) ... acc ...))]))
```

7.1.3 Maximum with an accumulator

```
;; (list-max lon) produces the largest element of lon.
;; list-max: (ne-listof Num) -> Num
(check-expect (list-max '(1 7 3)) 7)
(define (list-max lon) (max/acc (rest lon) (first lon)))

;; (max/acc lon best) produces the max of best and all elements of lon.
;; max/acc: (listof Num) Num -> Num
(define (max/acc lon best)
  (cond [(empty? lon) best]
        [(> (first lon) best) (max/acc (rest lon) (first lon))]
        [else (max/acc (rest lon) best)]))
```

7.1.4 Average in one pass

Two accumulators can track several quantities at once:

```
;; (average lon) produces the mean of lon.
;; average: (ne-listof Num) -> Num
(check-expect (average '(1 2 3 6)) 3)
(define (average lon) (avg/acc lon 0 0))

;; (avg/acc lon total count): total and count are the sum and number
;;   of elements already processed.
;; avg/acc: (listof Num) Num Nat -> Num
(define (avg/acc lon total count)
  (cond [(empty? lon) (/ total count)]
        [else (avg/acc (rest lon) (+ total (first lon)) (add1 count))]))
```

> Accumulative functions are often *tail recursive*: the recursive call is the whole answer, so nothing waits to be done after it returns. In the stepper the expression stays the same size, unlike `(+ 2 (+ 3 (sum ...)))`, which grows. This matters for space efficiency in real implementations.

7.1.5 Context-dependent processing

Accumulators also give the helper access to context such as the position of an element or the previous element:

```
;; (increasing? lon) produces true if each element is larger than the
;;   one before it.
;; increasing?: (listof Num) -> Bool
(check-expect (increasing? '(1 3 8)) true)
(check-expect (increasing? '(1 3 3)) false)
(define (increasing? lon)
  (or (empty? lon) (inc/acc (rest lon) (first lon))))

;; inc/acc: (listof Num) Num -> Bool
(define (inc/acc lon prev)
  (cond [(empty? lon) true]
        [else (and (> (first lon) prev)
                   (inc/acc (rest lon) (first lon)))]))
```

7.2 Mutual recursion

Mutual recursion arises naturally from mutually recursive data definitions (Chapter 10), but even simple examples exist:

```
;; (is-even? n) produces true if n is even.
;; is-even?: Nat -> Bool
(define (is-even? n)
  (cond [(zero? n) true]
        [else (is-odd? (sub1 n))]))

;; (is-odd? n) produces true if n is odd.
;; is-odd?: Nat -> Bool
(define (is-odd? n)
```

```
  (cond [(zero? n) false]
        [else (is-even? (sub1 n))]))
```

Example 7.1 (Alternating elements)**.** Keep the elements in odd positions (1st, 3rd, . . .):

```
;; keep-alternates: (listof Any) -> (listof Any)
(check-expect (keep-alternates '(a b c d e)) '(a c e))
(define (keep-alternates lst)
  (cond [(empty? lst) empty]
        [else (cons (first lst) (skip-one (rest lst)))]))

;; skip-one: (listof Any) -> (listof Any)
(define (skip-one lst)
  (cond [(empty? lst) empty]
        [else (keep-alternates (rest lst))]))
```

Each function corresponds to a "state": keeping or skipping.

7.3 Generative recursion (preview)

In generative recursion the argument to the recursive call is generated by some computation, e.g. `(gcd m (remainder n m))`. Termination is no longer automatic and requires an argument. Chapter 13 covers this in depth.

7.4 Choosing a kind of recursion

- Start with structural recursion; it is simplest and terminates automatically.
- Add an accumulator when the answer depends on what came *before* (context), or to avoid repeated work like `append` in reverse.
- Use mutual recursion when the data definition is mutually recursive or the problem has distinct states.
- Use generative recursion only when the problem's structure, not the data's, dictates the decomposition.

7.5 Summary

- Accumulators carry information about the already-processed part; document what they mean.
- A wrapper supplies the initial accumulator.
- Accumulative reverse is $\Theta(n)$ versus $\Theta(n^2)$ structural.
- Mutual recursion: functions that call each other, often mirroring states or data definitions.

Chapter 8

Natural Numbers and Processing Multiple Lists

8.1 Natural numbers as recursive data

Lists are built from `empty` using `cons`. Similarly, natural numbers can be viewed as built from `0` using `add1`:

```
;; A Nat is one of:
;; * 0
;; * (add1 Nat)
```

The analogues of `empty?` and `rest` are `zero?` and `sub1`. This gives a template:

```
;; nat-template: Nat -> Any
(define (nat-template n)
  (cond [(zero? n) ...]
        [else (... n ... (nat-template (sub1 n)) ...)]))
```

Example 8.1 (Countdown).

```
;; (countdown n) produces the list n, n-1, ..., 1, 0.
;; Examples:
(check-expect (countdown 0) '(0))
(check-expect (countdown 3) '(3 2 1 0))

;; countdown: Nat -> (listof Nat)
(define (countdown n)
  (cond [(zero? n) (cons 0 empty)]
        [else (cons n (countdown (sub1 n)))]))
```

Example 8.2 (Factorial).

```
;; (factorial n) produces n! = 1 * 2 * ... * n.
;; factorial: Nat -> Nat
(check-expect (factorial 0) 1)
(check-expect (factorial 5) 120)
(define (factorial n)
  (cond [(zero? n) 1]
        [else (* n (factorial (sub1 n)))]))
```

Condensed trace:

```
(factorial 3)
=> (* 3 (factorial 2))
=> (* 3 (* 2 (factorial 1)))
=> (* 3 (* 2 (* 1 (factorial 0))))
=> (* 3 (* 2 (* 1 1)))
=> 6
```

8.1.1 Subintervals of the naturals

To count down to a lower bound `b` instead of 0, change the base case:

```
;; An integer n >= b is one of:
;; * b
;; * (add1 m) where m >= b

;; (countdown-to n b) produces n, n-1, ..., b.
;; countdown-to: Int Int -> (listof Int)
;;   requires: n >= b
(check-expect (countdown-to 5 3) '(5 4 3))
(check-expect (countdown-to -1 -2) '(-1 -2))
(define (countdown-to n b)
  (cond [(= n b) (cons b empty)]
        [else (cons n (countdown-to (sub1 n) b))]))
```

Counting *up* uses `add1` toward an upper bound:

```
;; (countup-to n b) produces n, n+1, ..., b.
;; countup-to: Int Int -> (listof Int)
;;   requires: n <= b
(check-expect (countup-to 2 5) '(2 3 4 5))
(define (countup-to n b)
  (cond [(= n b) (cons b empty)]
        [else (cons n (countup-to (add1 n) b))]))
```

Example 8.3 (Primality by trial division)**.**

```
;; (prime? n) produces true if n is prime.
;; prime?: Nat -> Bool
(check-expect (prime? 7) true)
(check-expect (prime? 9) false)
(check-expect (prime? 1) false)
(define (prime? n)
  (and (> n 1) (no-divisors? n (sub1 n))))

;; (no-divisors? n k) produces true if no integer in 2..k divides n.
;; no-divisors?: Nat Nat -> Bool
(define (no-divisors? n k)
  (cond [(<= k 1) true]
        [else (and (not (zero? (remainder n k)))
                   (no-divisors? n (sub1 k)))]))
```

8.2 Processing two lists

When a function consumes two lists, there are three typical situations.

8.2.1 Case 1: one list goes along for the ride

Only one list is recursed on; the other is passed unchanged. `my-append` from the previous chapter is the standard example. The template is the ordinary list template with an extra parameter.

8.2.2 Case 2: processing in lockstep

The lists have the same length, and corresponding elements are processed together.

```
;; (dot-product l1 l2) produces the dot product of l1 and l2.
;; Examples:
(check-expect (dot-product '(1 2 3) '(4 5 6)) 32)
(check-expect (dot-product empty empty) 0)

;; dot-product: (listof Num) (listof Num) -> Num
;;   requires: l1 and l2 have the same length
(define (dot-product l1 l2)
  (cond [(empty? l1) 0]
        [else (+ (* (first l1) (first l2))
                 (dot-product (rest l1) (rest l2)))]))
```

Since both lists empty out simultaneously, checking one suffices.

8.2.3 Case 3: processing at different rates

In general, either list can be empty independently, giving four combinations.

```
;; twolist-template: (listof X) (listof Y) -> Any
(define (twolist-template lx ly)
  (cond [(and (empty? lx) (empty? ly)) ...]
        [(and (empty? lx) (cons? ly))
         (... (first ly) ... (rest ly) ...)]
        [(and (cons? lx) (empty? ly))
         (... (first lx) ... (rest lx) ...)]
        [(and (cons? lx) (cons? ly))
         ;; may recurse on (rest lx), (rest ly), or both
         (... (first lx) ... (first ly) ...
              (twolist-template (rest lx) ly) ...
              (twolist-template lx (rest ly)) ...
              (twolist-template (rest lx) (rest ly)) ...)]))
```

Often some cases collapse.

Example 8.4 (Merging sorted lists)**.**

```
;; (merge l1 l2) produces a sorted list of all elements of the sorted
;;   lists l1 and l2.
;; Examples:
(check-expect (merge '(1 4 9) '(2 3 10)) '(1 2 3 4 9 10))
(check-expect (merge empty '(5)) '(5))

;; merge: (listof Num) (listof Num) -> (listof Num)
;;   requires: l1 and l2 are sorted in non-decreasing order
(define (merge l1 l2)
  (cond [(empty? l1) l2]
        [(empty? l2) l1]
```

```
        [(< (first l1) (first l2))
         (cons (first l1) (merge (rest l1) l2))]
        [else (cons (first l2) (merge l1 (rest l2)))]))
```

Condensed trace:

```
(merge (list 1 4) (list 2 3))
=> (cons 1 (merge (list 4) (list 2 3)))
=> (cons 1 (cons 2 (merge (list 4) (list 3))))
=> (cons 1 (cons 2 (cons 3 (merge (list 4) empty))))
=> (cons 1 (cons 2 (cons 3 (list 4))))
=> (list 1 2 3 4)
```

Example 8.5 (List equality).

```
;; (list=? l1 l2) produces true if l1 and l2 contain the same numbers
;;   in the same order.
;; list=?: (listof Num) (listof Num) -> Bool
(check-expect (list=? '(1 2) '(1 2)) true)
(check-expect (list=? '(1 2) '(1)) false)
(define (list=? l1 l2)
  (cond [(and (empty? l1) (empty? l2)) true]
        [(or (empty? l1) (empty? l2)) false]
        [else (and (= (first l1) (first l2))
                   (list=? (rest l1) (rest l2)))]))
```

8.2.4 Consuming a list and a number

A list and a `Nat` can be processed together, e.g. `list-ref`:

```
;; (my-list-ref lst n) produces the element at position n (0-based),
;;   or false if lst is too short.
;; my-list-ref: (listof Any) Nat -> (anyof Any false)
(check-expect (my-list-ref '(a b c) 1) 'b)
(check-expect (my-list-ref '(a) 3) false)
(define (my-list-ref lst n)
  (cond [(empty? lst) false]
        [(zero? n) (first lst)]
        [else (my-list-ref (rest lst) (sub1 n))]))
```

8.3 Summary

- Natural numbers have a recursive data definition (`0` or `add1`); template uses `zero?` and `sub1`.
- Intervals: change the base case to a bound `b`; counting up uses `add1`.
- Two lists: along for the ride, lockstep, or at different rates (four-case template).
- `merge` of sorted lists is the key example of different-rate processing.

Chapter 9

Patterns of Recursion and Dictionaries

This chapter consolidates the recursive patterns seen so far and applies them to a very common abstract data type: the **dictionary**.

9.1 A catalogue of patterns

Most list functions written so far fall into one of a few shapes. Recognizing the shape tells you most of the code before you write it.

Pattern	Base case	Recursive case
Combine (sum, count)	identity value	`(op (f (first l)) (recur (rest l)))`
Search (contains?)	`false`	`(or (test (first l)) (recur (rest l)))`
All-satisfy	`true`	`(and (test (first l)) (recur (rest l)))`
Transform (map)	`empty`	`(cons (f (first l)) (recur (rest l)))`
Select (filter)	`empty`	`cons` first only if it passes the test
Accumulate	accumulator	recur on rest with updated accumulator
Find-first	`false`	first if it passes, else recur

These shapes are exactly what higher-order functions (`foldr`, `ormap`, `andmap`, `map`, `filter`, `foldl`) capture in Chapter 12.

9.2 Dictionaries

Definition 9.1 (Dictionary)**.** A **dictionary** (also map, table, associative array) stores *key–value pairs* where each key occurs at most once. Operations:

- **lookup**: given a key, produce its value (or indicate absence);
- **add**: add a key and value (or replace the value of an existing key);
- **remove**: delete the pair with a given key.

Examples: student ID → name, word → definition, symbol → count. The dictionary is an **abstract data type** (ADT): it is defined by its operations, and may be implemented in different ways with different efficiency.

9.3 Association lists

The simplest implementation is a list of two-element lists.

```
;; An Association List (AL) is one of:
;; * empty
;; * (cons (list Num Str) AL)
;;   requires: each key (Num) is unique

(define marks '((12345 "Ada") (31415 "Grace") (27182 "Alan")))
```

Template.

```
;; al-template: AL -> Any
(define (al-template alst)
  (cond [(empty? alst) ...]
        [else (... (first (first alst)) ...     ; key
                   (second (first alst)) ...    ; value
                   (al-template (rest alst)) ...)]))
```

9.3.1 Lookup

```
;; (lookup-al k alst) produces the value associated with k in alst,
;;   or false if k is not a key.
;; Examples:
(check-expect (lookup-al 31415 marks) "Grace")
(check-expect (lookup-al 99999 marks) false)

;; lookup-al: Num AL -> (anyof Str false)
(define (lookup-al k alst)
  (cond [(empty? alst) false]
        [(= k (first (first alst))) (second (first alst))]
        [else (lookup-al k (rest alst))]))
```

Producing `false` for "not found" is a common CS 135 convention; the contract uses `(anyof Str false)` to document it.

Trace.

```
(lookup-al 3 (list (list 1 "a") (list 3 "c")))
=> (cond [(empty? (list (list 1 "a") (list 3 "c"))) false] ...)
...  ; first clause false, second: (= 3 1) is false
=> (lookup-al 3 (rest (list (list 1 "a") (list 3 "c"))))
=> (lookup-al 3 (list (list 3 "c")))
...  ; (= 3 3) is true
=> (second (first (list (list 3 "c"))))
=> (second (list 3 "c"))
=> "c"
```

9.3.2 Add or update

```
;; (add-al k v alst) produces an AL like alst but with k mapped to v,
;;   replacing any previous value for k.
;; Examples:
(check-expect (add-al 1 "x" empty) '((1 "x")))
(check-expect (add-al 1 "y" '((1 "x") (2 "z"))) '((1 "y") (2 "z")))
(check-expect (add-al 3 "w" '((1 "x"))) '((1 "x") (3 "w")))

;; add-al: Num Str AL -> AL
(define (add-al k v alst)
  (cond [(empty? alst) (list (list k v))]
        [(= k (first (first alst))) (cons (list k v) (rest alst))]
        [else (cons (first alst) (add-al k v (rest alst)))]))
```

9.3.3 Remove

```
;; (remove-al k alst) produces alst without the pair with key k.
;; remove-al: Num AL -> AL
(check-expect (remove-al 2 '((1 "a") (2 "b") (3 "c"))) '((1 "a") (3 "c")))
(check-expect (remove-al 9 '((1 "a"))) '((1 "a")))
(define (remove-al k alst)
  (cond [(empty? alst) empty]
        [(= k (first (first alst))) (rest alst)]
        [else (cons (first alst) (remove-al k (rest alst)))]))
```

9.3.4 Efficiency

Each operation may traverse the whole list, so all three take time proportional to n, the number of pairs. Keeping the list sorted by key does not help asymptotically for a linked list (we still have to walk to the position). Binary search trees (Chapter 10) bring lookup down to proportional to the tree's height.

9.4 Application: word frequencies

```
;; A FreqTable is a (listof (list Sym Nat)) with unique symbols.

;; (count-words los) produces a FreqTable giving how often each symbol
;;   occurs in los.
;; count-words: (listof Sym) -> FreqTable
(check-expect (count-words '(a b a)) '((a 2) (b 1)))
(define (count-words los)
  (cond [(empty? los) empty]
        [else (increment (first los) (count-words (rest los)))]))

;; (increment s ft) adds 1 to the count of s in ft (adding s with
;;   count 1 if absent), keeping s at the front.
;; increment: Sym FreqTable -> FreqTable
(define (increment s ft)
  (cond [(empty? ft) (list (list s 1))]
        [(symbol=? s (first (first ft)))
```

```
         (cons (list s (add1 (second (first ft)))) (rest ft))]
        [else (cons (first ft) (increment s (rest ft)))]))
```

(Order of the result depends on processing order; tests should match the order your design specifies, or the purpose should promise none.)

9.5 Pairs: lists or structures?

A key–value pair could be `(list k v)` or `(make-entry k v)`. Lists are convenient with list abbreviations and quoting; structures give named selectors and a type predicate. CS 135 uses both; be consistent within a program and say which in the data definition.

9.6 Summary

- Many list functions follow a small number of recursive patterns.
- A dictionary maps unique keys to values with lookup, add, remove.
- Association lists implement dictionaries in linear time per operation.
- “Not found” is commonly signalled by producing `false`.

Chapter 10

Trees

Lists are linear: each element has one successor. **Trees** allow each node to have several children, which models hierarchies (file systems, organization charts), arithmetic expressions, and efficient search structures.

10.1 Terminology

A tree consists of **nodes**. One node is the **root**; every other node has exactly one **parent**. Nodes with no children are **leaves**. The **subtree** rooted at a node contains it and all its descendants. The **depth** of a node is the number of edges from the root; the **height** of a tree is the maximum depth of a leaf (conventions differ by one; CS 135 often counts nodes on the longest root-to-leaf path).

10.2 Binary trees

Each node has at most two children, a left and a right subtree, either of which may be empty.

```
(define-struct node (key left right))
;; A Node is a (make-node Num BT BT)

;; A binary tree (BT) is one of:
;; * empty
;; * Node
```

Template. Two self-references, so two recursive calls:

```
;; bt-template: BT -> Any
(define (bt-template t)
  (cond [(empty? t) ...]
        [else (... (node-key t) ...
                   (bt-template (node-left t)) ...
                   (bt-template (node-right t)) ...)]))
```

```
;; (size t) produces the number of nodes in t.
;; size: BT -> Nat
(define (size t)
  (cond [(empty? t) 0]
```

```
        [else (+ 1 (size (node-left t)) (size (node-right t)))]))

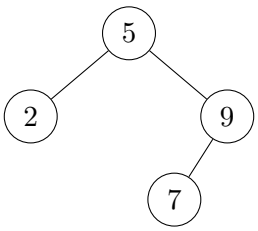
;; (height t) produces the number of nodes on a longest root-leaf path.
;; height: BT -> Nat
(define (height t)
  (cond [(empty? t) 0]
        [else (add1 (max (height (node-left t)) (height (node-right t))))]))

(define t1 (make-node 5 (make-node 2 empty empty)
                        (make-node 9 (make-node 7 empty empty) empty)))
(check-expect (size t1) 4)
(check-expect (height t1) 3)
```

10.3 Binary search trees

Definition 10.1 (BST ordering property)**.** A binary tree is a **binary search tree** if for every node with key k, every key in its left subtree is less than k and every key in its right subtree is greater than k.

Tree `t1` above is a BST:



For dictionaries, each node stores a key and a value:

```
(define-struct bnode (key val left right))
;; A BST is one of:
;; * empty
;; * (make-bnode Num Str BST BST)
;;   requires: keys in left are < key, keys in right are > key
```

10.3.1 Search

The ordering property means only *one* subtree need be searched:

```
;; (search-bst k t) produces the value for key k in t, or false.
;; search-bst: Num BST -> (anyof Str false)
(define (search-bst k t)
  (cond [(empty? t) false]
        [(= k (bnode-key t)) (bnode-val t)]
        [(< k (bnode-key t)) (search-bst k (bnode-left t))]
        [else (search-bst k (bnode-right t))]))

(define dict (make-bnode 5 "five"
               (make-bnode 2 "two" empty empty)
               (make-bnode 9 "nine" empty empty)))
(check-expect (search-bst 9 dict) "nine")
(check-expect (search-bst 4 dict) false)
```

Condensed trace:

```
(search-bst 9 dict)
=> (search-bst 9 (make-bnode 9 "nine" empty empty))   ; 9 > 5: go right
=> "nine"
```

The work is proportional to the height of the tree: about $\log_2 n$ if the tree is balanced, but n in the worst case (a "stick" created by inserting keys in sorted order).

10.3.2 Insertion

```
;; (insert-bst k v t) produces a BST like t with k mapped to v.
;; insert-bst: Num Str BST -> BST
(check-expect (insert-bst 1 "one" empty) (make-bnode 1 "one" empty empty))
(define (insert-bst k v t)
  (cond [(empty? t) (make-bnode k v empty empty)]
        [(= k (bnode-key t))
         (make-bnode k v (bnode-left t) (bnode-right t))]
        [(< k (bnode-key t))
         (make-bnode (bnode-key t) (bnode-val t)
                     (insert-bst k v (bnode-left t)) (bnode-right t))]
        [else
         (make-bnode (bnode-key t) (bnode-val t)
                     (bnode-left t) (insert-bst k v (bnode-right t)))]))
```

Note this builds a *new* tree; only the nodes along the search path are reconstructed.

10.3.3 Traversal

An **in-order** traversal (left, root, right) of a BST lists keys in sorted order:

```
;; bst->list: BST -> (listof Num)
(define (bst->list t)
  (cond [(empty? t) empty]
        [else (append (bst->list (bnode-left t))
                      (list (bnode-key t))
                      (bst->list (bnode-right t)))]))
```

10.4 Evaluation (expression) trees

An arithmetic expression such as $2 \times (3 + 4)$ is naturally a tree with operators at internal nodes and numbers at leaves; the tree structure makes the order of operations explicit, so no parentheses or precedence rules are needed.

```
(define-struct binode (op arg1 arg2))
;; A BinExp is one of:
;; * Num
;; * (make-binode (anyof '* '+ '/ '-) BinExp BinExp)

;; (eval-bin ex) produces the value of the expression ex.
;; eval-bin: BinExp -> Num
(check-expect (eval-bin 5) 5)
(check-expect (eval-bin (make-binode '* 2 (make-binode '+ 3 4))) 14)
```

```
(define (eval-bin ex)
  (cond [(number? ex) ex]
        [else (apply-op (binode-op ex)
                        (eval-bin (binode-arg1 ex))
                        (eval-bin (binode-arg2 ex)))]))

;; apply-op: Sym Num Num -> Num
(define (apply-op op a b)
  (cond [(symbol=? op '*) (* a b)]
        [(symbol=? op '+) (+ a b)]
        [(symbol=? op '/) (/ a b)]
        [(symbol=? op '-) (- a b)]))
```

10.5 General trees and mutual recursion

If an operator may have any number of arguments, as in Racket itself (`(+ 1 2 3 4)`), a node holds a *list* of subtrees. The data definitions become **mutually recursive**:

```
(define-struct opnode (op args))
;; An AExp is one of:
;; * Num
;; * (make-opnode (anyof '* '+) (listof AExp))
```

Here `AExp` refers to `(listof AExp)` and that list type refers back to `AExp`. The templates are likewise mutually recursive — one function per type:

```
;; aexp-template: AExp -> Any
(define (aexp-template ex)
  (cond [(number? ex) ...]
        [else (... (opnode-op ex) ...
                   (listof-aexp-template (opnode-args ex)) ...)]))

;; listof-aexp-template: (listof AExp) -> Any
(define (listof-aexp-template exlist)
  (cond [(empty? exlist) ...]
        [else (... (aexp-template (first exlist)) ...
                   (listof-aexp-template (rest exlist)) ...)]))
```

```
;; (eval ex) produces the value of ex.
;; eval: AExp -> Num
(check-expect (eval (make-opnode '+ (list 1 (make-opnode '* (list 2 3 4)))))
              25)
(define (eval ex)
  (cond [(number? ex) ex]
        [else (apply (opnode-op ex) (opnode-args ex))]))

;; (apply f exlist) applies operator f to the values of exlist.
;; apply: Sym (listof AExp) -> Num
(define (apply f exlist)
  (cond [(empty? exlist) (cond [(symbol=? f '*) 1] [else 0])]
        [(symbol=? f '*) (* (eval (first exlist)) (apply f (rest exlist)))]
        [else (+ (eval (first exlist)) (apply f (rest exlist)))]))
```

The base case uses the identity of each operator: the empty sum is 0 and the empty product is 1. (These names clash with built-ins in Intermediate Student; rename to `my-eval`/`my-apply` there.)

10.5.1 Condensed trace

```
(eval (make-opnode '+ (list 1 (make-opnode '* (list 2 3)))))
=> (apply '+ (list 1 (make-opnode '* (list 2 3))))
=> (+ (eval 1) (apply '+ (list (make-opnode '* (list 2 3)))))
=> (+ 1 (apply '+ (list (make-opnode '* (list 2 3)))))
=> (+ 1 (+ (eval (make-opnode '* (list 2 3))) (apply '+ empty)))
=> (+ 1 (+ (apply '* (list 2 3)) (apply '+ empty)))
=> (+ 1 (+ (* 2 (* 3 (apply '* empty))) (apply '+ empty)))
=> (+ 1 (+ (* 2 (* 3 1)) 0))
=> 7
```

10.6 Nested lists and leaf-labelled trees

A nested list such as `'(1 (2 3) ((4)))` is a tree whose leaves are the numbers:

```
;; A Nested-List-Num is one of:
;; * empty
;; * (cons Num Nested-List-Num)
;; * (cons Nested-List-Num Nested-List-Num)

;; (flatten lst) produces the numbers in lst in order, without nesting.
;; flatten: Nested-List-Num -> (listof Num)
(check-expect (flatten '(1 (2 3) ((4)))) '(1 2 3 4))
(define (flatten lst)
  (cond [(empty? lst) empty]
        [(number? (first lst)) (cons (first lst) (flatten (rest lst)))]
        [else (append (flatten (first lst)) (flatten (rest lst)))]))
```

10.7 Summary

- Binary trees: two self-references, so two recursive calls in the template.
- BSTs: the ordering property lets search and insert follow one path; time proportional to height.
- Expression trees: operators at internal nodes, numbers at leaves; evaluation is a recursive traversal.
- General trees use a list of children, producing mutually recursive data definitions and templates.

Chapter 11

Local Definitions and Lexical Scope

From this point on we use **Intermediate Student**, which adds the `local` special form. `local` lets us make definitions visible only inside one expression.

11.1 The `local` form

```
(local [(define x1 e1)
        ...
        (define xn en)]
  body)
```

The definitions (constants or functions) are visible in `body` and in each other's definitions (later definitions may use earlier ones), but not outside the `local` expression.

11.2 Uses of local

11.2.1 1. Clarity: naming intermediate values

Heron's formula for the area of a triangle with sides a, b, c: $A = \sqrt{s(s-a)(s-b)(s-c)}$ where $s = (a+b+c)/2$.

```
;; (heron a b c) produces the area of a triangle with side lengths a, b, c.
;; heron: Num Num Num -> Num
;;   requires: a, b, c satisfy the triangle inequality
(check-expect (heron 3 4 5) 6)
(define (heron a b c)
  (local [(define s (/ (+ a b c) 2))]
    (sqrt (* s (- s a) (- s b) (- s c)))))
```

11.2.2 2. Efficiency: avoiding repeated computation

```
;; list-max: (ne-listof Num) -> Num
(define (list-max lon)
  (cond [(empty? (rest lon)) (first lon)]
        [else (local [(define max-rest (list-max (rest lon)))]
                (cond [(> (first lon) max-rest) (first lon)]
                      [else max-rest]))]))
```

Without `local`, writing `(list-max (rest lon))` in both the question and the answer makes two recursive calls per level, i.e. $\Theta(2^n)$ time. With `local` it is evaluated once: $\Theta(n)$.

11.2.3 3. Encapsulation: hiding helper functions

Helpers that are only meaningful to one function can be hidden inside it, for example the accumulator helper of reverse:

```
;; (my-reverse lst) produces lst in reverse order.
;; my-reverse: (listof Any) -> (listof Any)
(check-expect (my-reverse '(1 2 3)) '(3 2 1))
(define (my-reverse lst)
  (local [;; (rev/acc l acc) produces l reversed, followed by acc
          ;; rev/acc: (listof Any) (listof Any) -> (listof Any)
          (define (rev/acc l acc)
            (cond [(empty? l) acc]
                  [else (rev/acc (rest l) (cons (first l) acc))]))]
    (rev/acc lst empty)))
```

Locally defined functions need a purpose and contract, but no separate examples or tests (they cannot be called from outside, so they are tested through the enclosing function).

11.2.4 4. Using enclosing parameters

A local function can refer to parameters of the enclosing function, so they need not be passed along explicitly:

```
;; (count-above lon t) produces how many elements of lon exceed t.
;; count-above: (listof Num) Num -> Nat
(define (count-above lon t)
  (local [;; count/l: (listof Num) -> Nat
          (define (count/l l)
            (cond [(empty? l) 0]
                  [(> (first l) t) (add1 (count/l (rest l)))]
                  [else (count/l (rest l))]))]
    (count/l lon)))
```

This becomes essential with `lambda` in the next chapter.

11.3 Semantics of local

The substitution model is extended with a rule that **renames** each locally defined name to a fresh, unique name and **lifts** the definitions to the top level.

Local rule

`(local [(define x1 e1) ... (define xn en)] body)` ⇒ `(define x1' e1') ... (define xn' en') body'`, where each `xi'` is a fresh identifier and `e1'`, ..., `body'` are the original expressions with every `xi` replaced by `xi'`. The lifted definitions are then evaluated in order like top-level definitions.

Example 11.1 (Trace of local)**.**

```
(heron 3 4 5)
=> (local [(define s (/ (+ 3 4 5) 2))]
     (sqrt (* s (- s 3) (- s 4) (- s 5))))
=> (define s_0 (/ (+ 3 4 5) 2))
   (sqrt (* s_0 (- s_0 3) (- s_0 4) (- s_0 5)))
=> (define s_0 (/ 12 2))
   (sqrt (* s_0 (- s_0 3) (- s_0 4) (- s_0 5)))
=> (define s_0 6)
   (sqrt (* s_0 (- s_0 3) (- s_0 4) (- s_0 5)))
=> (define s_0 6)
   (sqrt (* 6 (- s_0 3) (- s_0 4) (- s_0 5)))
=> ...
=> (define s_0 6)
   (sqrt (* 6 3 2 1))
=> (define s_0 6)
   (sqrt 36)
=> (define s_0 6)
   6
```

Every call to `heron` creates a new fresh name (`s_0`, `s_1`, ...), so values from different calls never collide. In traces, previously evaluated lifted definitions can be shown once and then omitted.

11.4 Lexical scope

Definition 11.2 (Binding occurrence, scope)**.** A **binding occurrence** of an identifier is where it is introduced (a `define`, a function parameter, a `local` definition). The **scope** of a binding is the region of the program where uses of the name refer to that binding.

Racket uses **lexical (static) scope**: the binding a name refers to is determined by the program text. To find it, start at the use and move outward through enclosing `local`s and function headers until a binding for that name is found; if none, it is a top-level (global) binding.

Example 11.3 (Shadowing)**.**

```
(define x 1)
(define (f x)
  (local [(define x 10)
          (define (g y) (+ x y))]
    (g x)))
(f 5)   ; => 20
```

Inside `g`, `x` refers to the innermost binding, the local `x` = 10, which **shadows** the parameter `x` = 5 and the global `x` = 1. The argument in `(g x)` is also the local `x`. So the result is $10 + 10 = 20$.

It helps to draw **contour diagrams**: a box around each scope (top level, each function body, each `local`); a use refers to the nearest enclosing box that binds the name.

11.5 Style

- Use `local` to name values that are used more than once or that have a meaningful name.
- Do not over-nest: if a helper is useful on its own, keep it at top level and test it.

- Avoid reusing the same name at different levels (shadowing) unless deliberate; it confuses readers.

11.6 Summary

- `local` provides definitions visible in a single expression.
- Uses: clarity, efficiency (avoid recomputation), encapsulation, and access to enclosing parameters.
- Semantics: rename to fresh names and lift to the top level.
- Racket is lexically scoped; inner bindings shadow outer ones.

Chapter 12

Functional Abstraction

In **Intermediate Student with lambda**, functions are **first-class values**: they can be passed as arguments, produced as results, stored in lists and structures, and created without a name. This lets us abstract the recurring recursive patterns into reusable **higher-order functions**.

12.1 Functions as arguments

Compare two functions from Chapter 6:

```
(define (keep-positive lon)
  (cond [(empty? lon) empty]
        [(positive? (first lon)) (cons (first lon) (keep-positive (rest lon)))]
        [else (keep-positive (rest lon))]))

(define (keep-even lon)
  (cond [(empty? lon) empty]
        [(even? (first lon)) (cons (first lon) (keep-even (rest lon)))]
        [else (keep-even (rest lon))]))
```

They differ only in the predicate. Make the predicate a parameter:

```
;; (my-filter pred? lst) produces the elements of lst satisfying pred?.
;; my-filter: (X -> Bool) (listof X) -> (listof X)
(check-expect (my-filter even? '(1 2 3 4)) '(2 4))
(define (my-filter pred? lst)
  (cond [(empty? lst) empty]
        [(pred? (first lst)) (cons (first lst) (my-filter pred? (rest lst)))]
        [else (my-filter pred? (rest lst))]))
```

The contract uses a **type variable** `X`: `my-filter` works for any element type, as long as the predicate consumes that type. Contracts with type variables are called **generic** or **polymorphic**.

12.2 lambda: anonymous functions

Defining a named function just to pass it once is clumsy. `(lambda (x1 ... xn) body)` produces a function value directly:

```
(my-filter (lambda (n) (> n 10)) '(5 15 25))   ; => '(15 25)
((lambda (x y) (+ x (* 2 y))) 1 3)              ; => 7
```

In fact `(define (f x) body)` is shorthand for `(define f (lambda (x) body))`. A function is just a value bound to a name.

Substitution rule for lambda

`((lambda (x1 ... xn) exp) v1 ... vn)` ⇒ `exp'` where `exp'` is `exp` with each `xi` replaced by `vi`. A `lambda` expression is itself a value.

12.3 Functions as results

```
;; (make-adder n) produces a function that adds n to its argument.
;; make-adder: Num -> (Num -> Num)
(define (make-adder n)
  (lambda (x) (+ x n)))

(define add5 (make-adder 5))
(check-expect (add5 10) 15)
(check-expect ((make-adder 1) 1) 2)
```

Trace:

```
((make-adder 1) 1)
=> ((lambda (x) (+ x 1)) 1)
=> (+ 1 1)
=> 2
```

The produced function "remembers" `n`; such a function together with the bindings it captures is called a **closure**. Lexical scope determines which `n` is meant. Combining `local` and `lambda` gives the same effect with a named helper.

12.4 The built-in abstract list functions

Function	Contract
`map`	`(X -> Y) (listof X) -> (listof Y)`
`filter`	`(X -> Bool) (listof X) -> (listof X)`
`foldr`	`(X Y -> Y) Y (listof X) -> Y`
`foldl`	`(X Y -> Y) Y (listof X) -> Y`
`build-list`	`Nat (Nat -> X) -> (listof X)`
`andmap`, `ormap`	`(X -> Bool) (listof X) -> Bool`
`apply`	`(X ... -> Y) (listof X) -> Y`

12.4.1 map

`(map f (list x1 ... xn))` = `(list (f x1) ... (f xn))`.

```
(map sqr '(1 2 3))                          ; => '(1 4 9)
(map (lambda (s) (string-append s "!")) '("hi" "yo"))  ; => '("hi!" "yo!")
(map + '(1 2) '(10 20))                     ; => '(11 22)  (several lists)
```

12.4.2 foldr

`foldr` abstracts the "combine" pattern of structural recursion:

`(foldr f base (list x1 x2 ... xn)) = (f x1 (f x2 ... (f xn base)))`

```
;; my-foldr: (X Y -> Y) Y (listof X) -> Y
(define (my-foldr combine base lst)
  (cond [(empty? lst) base]
        [else (combine (first lst) (my-foldr combine base (rest lst)))]))
```

The first argument of `combine` is the current element; the second is the result of folding the rest. Many functions are one-liners:

```
(define (sum lon) (foldr + 0 lon))
(define (count lst) (foldr (lambda (x rr) (add1 rr)) 0 lst))
(define (my-map f lst) (foldr (lambda (x rr) (cons (f x) rr)) empty lst))
(define (my-append l1 l2) (foldr cons l2 l1))
```

Trace:

```
(foldr + 0 (list 1 2 3))
=> (+ 1 (foldr + 0 (list 2 3)))
=> (+ 1 (+ 2 (foldr + 0 (list 3))))
=> (+ 1 (+ 2 (+ 3 (foldr + 0 empty))))
=> (+ 1 (+ 2 (+ 3 0)))
=> 6
```

12.4.3 foldl

`foldl` abstracts the *accumulator* pattern; it combines from the left:

`(foldl f base (list x1 ... xn)) = (f xn (... (f x2 (f x1 base))))`

```
;; my-foldl: (X Y -> Y) Y (listof X) -> Y
(define (my-foldl combine acc lst)
  (cond [(empty? lst) acc]
        [else (my-foldl combine (combine (first lst) acc) (rest lst))]))

(define (my-reverse lst) (foldl cons empty lst))
(check-expect (my-reverse '(1 2 3)) '(3 2 1))
```

12.4.4 build-list

`(build-list n f)` = `(list (f 0) (f 1) ... (f (sub1 n)))`; it abstracts recursion on `Nat`.

```
(build-list 5 sqr)                    ; => '(0 1 4 9 16)
(define (countup-to n b) (build-list (add1 (- b n)) (lambda (i) (+ n i))))
```

12.4.5 Combining abstractions

```
;; (sum-of-even-squares lon) produces the sum of squares of even
;;   elements of lon.
;; sum-of-even-squares: (listof Int) -> Int
(check-expect (sum-of-even-squares '(1 2 3 4)) 20)
(define (sum-of-even-squares lon)
  (foldr + 0 (map sqr (filter even? lon))))
```

12.5 Abstraction with local and lambda

Parameters of the enclosing function are visible inside a `lambda`:

```
;; (scale-all k lon) multiplies each element of lon by k.
;; scale-all: Num (listof Num) -> (listof Num)
(define (scale-all k lon) (map (lambda (x) (* k x)) lon))

;; (above-avg lon) keeps the elements greater than the average.
;; above-avg: (ne-listof Num) -> (listof Num)
(define (above-avg lon)
  (local [(define avg (/ (foldr + 0 lon) (length lon)))]
    (filter (lambda (x) (> x avg)) lon)))
```

12.6 Functional abstraction on other data

Abstraction works for any recursive data. A generic fold over binary trees:

```
;; bt-fold: (Num Y Y -> Y) Y BT -> Y
(define (bt-fold f base t)
  (cond [(empty? t) base]
        [else (f (node-key t)
                 (bt-fold f base (node-left t))
                 (bt-fold f base (node-right t)))]))

(define (size t) (bt-fold (lambda (k l r) (+ 1 l r)) 0 t))
(define (tree-sum t) (bt-fold (lambda (k l r) (+ k l r)) 0 t))
```

12.7 When to use what

- Each element transformed independently → `map`.
- Keep some elements → `filter`.
- Combine all elements into one value → `foldr` (or `foldl` for accumulator-style).
- Generate a list from indices $0..n-1$ → `build-list`.
- Test whether all/any satisfy a predicate → `andmap`/`ormap`.
- Processing that needs early stopping, two lists at different rates, or generative recursion usually needs explicit recursion.

12.8 Contracts for higher-order functions

- Function types are written in parentheses: `(Num -> Bool)`.
- Use type variables (`X`, `Y`) when the function is generic; the same variable must stand for the same type throughout a contract.
- Example: `compose: (Y -> Z) (X -> Y) -> (X -> Z)`.

12.9 Summary

- Functions are first-class values; `lambda` creates them anonymously.
- Abstracting a common pattern means turning the differing part into a function parameter.
- `map`, `filter`, `foldr`, `foldl`, `build-list` capture the main list recursion patterns.
- A function produced by another function captures the bindings it refers to (closure).
- Generic contracts use type variables.

Chapter 13

Generative Recursion

Definition 13.1 (Generative recursion)**.** A recursive function uses **generative recursion** if at least one recursive call is on a value that is *generated* by computation, rather than being a structural part of the input (like `(rest lst)` or `(sub1 n)`).

Generative recursion is more powerful but has two costs: the code no longer follows from a template, and **termination** is no longer guaranteed. The design recipe is modified: think of the algorithm first (how to generate smaller subproblems and combine their solutions), and argue termination.

13.1 Euclid's algorithm

For $m, n \in \mathbb{N}$ with $n > 0$: $\gcd(m, 0) = m$ and $\gcd(m, n) = \gcd(n, m \bmod n)$.

```
;; (euclid-gcd m n) produces the greatest common divisor of m and n.
;; Examples:
(check-expect (euclid-gcd 24 18) 6)
(check-expect (euclid-gcd 7 0) 7)

;; euclid-gcd: Nat Nat -> Nat
(define (euclid-gcd m n)
  (cond [(zero? n) m]
        [else (euclid-gcd n (remainder m n))]))
```

Trace.

```
(euclid-gcd 24 18)
=> (euclid-gcd 18 6)      ; (remainder 24 18) = 6
=> (euclid-gcd 6 0)       ; (remainder 18 6) = 0
=> 6
```

Correctness. Any common divisor of m and n divides $m - qn = m \bmod n$, and conversely any common divisor of n and $m \bmod n$ divides m. So $\gcd(m, n) = \gcd(n, m \bmod n)$.

Termination. The second argument strictly decreases ($0 \leq m \bmod n < n$) and is a natural number, so it reaches 0 after finitely many steps. (In fact after at most about $2 \log_2 n$ steps, since the second argument at least halves every two calls.)

Termination arguments

To prove a generative recursion terminates, find a **measure**: a natural number computed from the arguments that strictly decreases at every recursive call. Since naturals cannot decrease forever, recursion must stop.

13.1.1 A function that may not terminate

The Collatz function:

```
;; collatz: Nat -> Nat     requires: n >= 1
(define (collatz n)
  (cond [(= n 1) 1]
        [(even? n) (collatz (/ n 2))]
        [else (collatz (+ 1 (* 3 n)))]))
```

No one knows whether this terminates for every $n \geq 1$ — it is a famous open problem. This illustrates why termination needs an argument.

13.2 Quicksort

Quicksort (C. A. R. Hoare, 1960) is a divide-and-conquer sorting algorithm:

1. If the list is empty, it is sorted.
2. Otherwise pick a **pivot** (the first element).
3. Partition the rest into elements less than the pivot and the others.
4. Sort each part recursively and append: smaller, pivot, larger.

```
;; (quick-sort lon) produces lon sorted in non-decreasing order.
;; Examples:
(check-expect (quick-sort empty) empty)
(check-expect (quick-sort '(3 1 4 1 5 9 2 6)) '(1 1 2 3 4 5 6 9))

;; quick-sort: (listof Num) -> (listof Num)
(define (quick-sort lon)
  (cond [(empty? lon) empty]
        [else
         (local [(define pivot (first lon))
                 (define less (filter (lambda (x) (< x pivot)) (rest lon)))
                 (define greater (filter (lambda (x) (>= x pivot)) (rest lon)))]
           (append (quick-sort less)
                   (list pivot)
                   (quick-sort greater)))]))
```

The recursive calls are on `less` and `greater`, lists computed by `filter` — this is generative recursion.

Condensed trace.

```
(quick-sort (list 3 5 1 4))
=> (append (quick-sort (list 1)) (list 3) (quick-sort (list 5 4)))
=> (append (append (quick-sort empty) (list 1) (quick-sort empty))
           (list 3) (quick-sort (list 5 4)))
=> (append (list 1) (list 3) (quick-sort (list 5 4)))
=> (append (list 1) (list 3)
           (append (quick-sort (list 4)) (list 5) (quick-sort empty)))
=> (append (list 1) (list 3) (append (list 4) (list 5) empty))
=> (list 1 3 4 5)
```

Termination. Measure: length of the list. Both `less` and `greater` are sublists of `(rest lon)`, so each has length strictly less than `(length lon)`.

Efficiency. Partitioning costs time proportional to the list length. If the pivot splits the list roughly in half each time, there are about $\log_2 n$ levels, each with $\Theta(n)$ total work: $\Theta(n \log n)$. If the list is already sorted, `less` is always empty and `greater` has length $n-1$: $n+(n-1)+\cdots = \Theta(n^2)$. A common bug is putting elements equal to the pivot in neither part (losing duplicates) or in a part with the pivot itself (non-termination).

13.3 Mergesort

Mergesort splits the list in half, sorts each half recursively, and uses `merge` from Chapter 8. It is always $\Theta(n \log n)$.

```
;; merge-sort: (listof Num) -> (listof Num)
(define (merge-sort lon)
  (cond [(or (empty? lon) (empty? (rest lon))) lon]
        [else
         (local [(define half (quotient (length lon) 2))]
           (merge (merge-sort (take lon half))
                  (merge-sort (drop lon half))))]))

;; (take lst n) produces the first n elements; (drop lst n) the rest.
;; take: (listof X) Nat -> (listof X)
(define (take lst n)
  (cond [(zero? n) empty]
        [else (cons (first lst) (take (rest lst) (sub1 n)))]))
;; drop: (listof X) Nat -> (listof X)
(define (drop lst n)
  (cond [(zero? n) lst]
        [else (drop (rest lst) (sub1 n))]))
```

Termination: for lists of length ≥ 2, both halves are strictly shorter.

13.4 Other examples of generative recursion

13.4.1 Binary search for a root (bisection)

To find x with $f(x) \approx 0$ on $[lo, hi]$ where $f(lo)$ and $f(hi)$ have opposite signs, repeatedly halve the interval:

```
;; (find-root f lo hi tol) produces an approximate root of f in [lo, hi].
;; find-root: (Num -> Num) Num Num Num -> Num
;;   requires: (f lo) and (f hi) have opposite signs; tol > 0
(define (find-root f lo hi tol)
  (local [(define mid (/ (+ lo hi) 2))]
    (cond [(< (- hi lo) tol) mid]
          [(<= (* (f lo) (f mid)) 0) (find-root f lo mid tol)]
          [else (find-root f mid hi tol)])))

(check-within (find-root (lambda (x) (- (* x x) 2)) 0 2 0.0001)
              (sqrt 2) 0.001)
```

Termination: the interval width halves each call, so after $\lceil \log_2((hi - lo)/tol) \rceil$ calls it is below the tolerance.

13.4.2 Processing strings in chunks

Splitting a list of characters into words, where each recursive call is on the characters after the next space, is generative: the amount removed depends on the data.

13.5 Design recipe for generative recursion

1. Identify trivially solvable cases and their solutions.
2. Describe how to generate one or more smaller problems.
3. Describe how to combine their solutions.
4. Give a termination argument (a decreasing measure) in a comment.
5. Test the trivial cases and cases that exercise each kind of split; include inputs that might cause non-termination (duplicates, sorted input).

13.6 Summary

- Generative recursion recurses on computed values; it is not driven by a template.
- Termination must be argued with a decreasing natural-number measure.
- Euclid's gcd, quicksort, mergesort and bisection are the canonical examples.
- Quicksort is $\Theta(n \log n)$ on average and $\Theta(n^2)$ in the worst case; mergesort is always $\Theta(n \log n)$.

Chapter 14

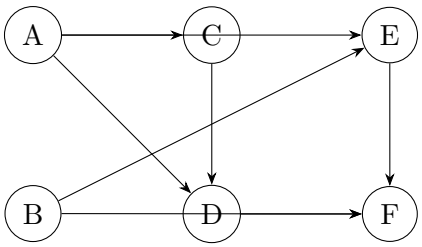
Graphs

Trees forbid a node from having more than one parent and forbid cycles. **Graphs** remove these restrictions and model road networks, social networks, course prerequisites, web links, and much more.

14.1 Definitions

Definition 14.1 (Directed graph)**.** A **directed graph** $G = (V, E)$ consists of a set V of **vertices** (nodes) and a set $E \subseteq V \times V$ of **edges**; an edge (v, w) goes *from* v *to* w.

- w is an **out-neighbour** of v if $(v, w) \in E$.
- A **path** is a sequence of vertices $v_1, v_2, \ldots, v_k$ with $(v_i, v_{i+1}) \in E$ for each i.
- A **cycle** is a path of length at least 2 (edges) with $v_1 = v_k$ (and otherwise distinct vertices).
- A graph with no cycles is **acyclic**; a directed acyclic graph is a **DAG**.



14.2 Representation: adjacency lists

Each vertex is paired with the list of its out-neighbours:

```
;; A Node is a Sym
;; A Graph is a (listof (list Node (listof Node)))
;;   requires: each node appears exactly once as the first element of an
;;             entry, and every out-neighbour is also a node of the graph.

(define g
  '((A (C D E))
    (B (E F))
```

```
    (C (D))
    (D (F))
    (E (F))
    (F ())))
```

Template.

```
;; graph-template: Graph -> Any
(define (graph-template G)
  (cond [(empty? G) ...]
        [else (... (first (first G)) ...                    ; Node
                   (listof-node-template (second (first G))) ...
                   (graph-template (rest G)) ...)]))
```

Neighbours.

```
;; (neighbours v G) produces the list of out-neighbours of v in G.
;; Examples:
(check-expect (neighbours 'A g) '(C D E))
(check-expect (neighbours 'F g) empty)

;; neighbours: Node Graph -> (listof Node)
;;   requires: v is a node in G
(define (neighbours v G)
  (cond [(symbol=? v (first (first G))) (second (first G))]
        [else (neighbours v (rest G))]))
```

14.3 Finding paths

Problem: given origin `orig` and destination `dest`, produce a path from `orig` to `dest` as a list of nodes, or `false` if none exists.

Idea. If `orig` equals `dest`, the path is `(list orig)`. Otherwise try to find a path from *some* neighbour of `orig` to `dest`; if one is found, put `orig` in front. This is mutual recursion between a function on a node and a function on a list of nodes, and it is generative: the neighbours are computed by `neighbours`.

```
;; (find-route orig dest G) finds a path from orig to dest in G, or
;;   produces false if there is none.
;; find-route: Node Node Graph -> (anyof (listof Node) false)
;;   requires: G is acyclic   (see below for the general version)
(define (find-route orig dest G)
  (cond [(symbol=? orig dest) (list orig)]
        [else (local [(define route (find-route/list (neighbours orig G) dest G))]
                (cond [(false? route) false]
                      [else (cons orig route)]))]))

;; (find-route/list los dest G) produces a path from some node in los
;;   to dest, or false if there is none.
;; find-route/list: (listof Node) Node Graph -> (anyof (listof Node) false)
(define (find-route/list los dest G)
  (cond [(empty? los) false]
```

```
        [else (local [(define route (find-route (first los) dest G))]
                (cond [(false? route) (find-route/list (rest los) dest G)]
                      [else route])))]))

(check-expect (find-route 'A 'F g) '(A C D F))
(check-expect (find-route 'C 'A g) false)
(check-expect (find-route 'E 'E g) '(E))
```

This strategy is called **backtracking**: explore one choice fully; if it fails, back up and try the next choice.

Condensed trace.

```
(find-route 'A 'F g)
=> ... (find-route/list '(C D E) 'F g) ...
=> ... (find-route 'C 'F g) ...
=> ... (find-route/list '(D) 'F g) ...
=> ... (find-route 'D 'F g) ...
=> ... (find-route/list '(F) 'F g) ...
=> ... (find-route 'F 'F g) ...            ; produces '(F)
=> '(A C D F)                                ; conses back up the chain
```

14.3.1 The problem with cycles

If the graph has a cycle, say we add the edge $D \to A$, then searching from `'A` for an unreachable node such as `'B` can go $A \to C \to D \to A \to C \to \cdots$ forever. The functions above do not terminate on cyclic graphs.

14.3.2 Fix: an accumulator of visited nodes

Keep a list of nodes already on the current path (or already visited) and never revisit them.

```
;; find-route: Node Node Graph -> (anyof (listof Node) false)
(define (find-route orig dest G)
  (find-route/acc orig dest G empty))

;; (find-route/acc orig dest G visited) finds a path from orig to dest
;;   that avoids the nodes in visited, or produces false.
;; find-route/acc: Node Node Graph (listof Node)
;;                 -> (anyof (listof Node) false)
(define (find-route/acc orig dest G visited)
  (cond [(symbol=? orig dest) (list orig)]
        [else (local [(define nbrs (neighbours orig G))
                      (define new-visited (cons orig visited))
                      (define route (find-route/list nbrs dest G new-visited))]
                (cond [(false? route) false]
                      [else (cons orig route)]))]))

;; find-route/list: (listof Node) Node Graph (listof Node)
;;                  -> (anyof (listof Node) false)
(define (find-route/list los dest G visited)
  (cond [(empty? los) false]
        [(member? (first los) visited)
         (find-route/list (rest los) dest G visited)]
```

```
        [else (local [(define route (find-route/acc (first los) dest G visited))]
                (cond [(false? route) (find-route/list (rest los) dest G visited)]
                      [else route])))]))
```

Termination. The measure is the number of graph nodes *not* in `visited`. Whenever `find-route/list` calls `find-route/acc`, it does so on a node not in `visited`, and that node is added before the next level, so the measure strictly decreases along every chain of calls. `find-route/list` itself recurses structurally on `los`.

Efficiency. With a visited list that only contains the current path, the search can still revisit the same node through different paths, which is exponential in the worst case. Passing back the set of *all* nodes explored so far (not just the current path) makes each node expanded at most once; this is the idea behind depth-first search covered in later courses (CS 341).

14.4 Other graph problems

- **Reachability**: the set of nodes reachable from a given node.
- **Topological sort** of a DAG (e.g. ordering courses to respect prerequisites).
- **Shortest paths** (breadth-first search, Dijkstra's algorithm) — later courses.

14.5 Summary

- A directed graph is a set of vertices and directed edges; represented as an adjacency list.
- Path finding by backtracking: mutual recursion between a node function and a list-of-nodes function.
- On cyclic graphs, a visited accumulator is needed for termination.
- Termination measure: number of unvisited nodes.

Chapter 15

Efficiency, Computing History, and What's Next

15.1 Measuring efficiency

Two correct programs can differ hugely in running time. Rather than timing programs (which depends on the machine), we count the number of substitution steps (or basic operations) as a function of the input size n, and care about how this grows for large n.

Definition 15.1 (Big-O, informally)**.** A function $T(n)$ is $\mathcal{O}(f(n))$ if there are constants $c > 0$ and n_0 such that $T(n) \le c\, f(n)$ for all $n \ge n_0$. We write $\Theta(f(n))$ when the bound is also a lower bound up to constants.

15.1.1 Typical running times in CS 135

Function	Running time
`first`, `rest`, `cons`, `empty?`	$\mathcal{O}(1)$
`length`, `member?`, `list-ref`, `sum`	$\mathcal{O}(n)$
`(append l1 l2)`	$\mathcal{O}$(length of $l1$)
`map`, `filter`, `foldr` (constant-time function)	$\mathcal{O}(n)$
association list lookup	$\mathcal{O}(n)$
BST search/insert	$\mathcal{O}(h)$; $\mathcal{O}(\log n)$ balanced, $\mathcal{O}(n)$ worst
insertion sort	$\mathcal{O}(n^2)$ worst, $\mathcal{O}(n)$ on sorted input
quicksort	$\mathcal{O}(n \log n)$ typical, $\mathcal{O}(n^2)$ worst
mergesort	$\mathcal{O}(n \log n)$
structural reverse with `append`	$\mathcal{O}(n^2)$
accumulative reverse	$\mathcal{O}(n)$
`list-max` with repeated recursive call	$\mathcal{O}(2^n)$
Euclid's gcd	$\mathcal{O}(\log n)$ calls

15.1.2 Analysing recursive functions

Write a **recurrence** for the running time and solve it:

- $T(n) = T(n-1) + \mathcal{O}(1) \Rightarrow \mathcal{O}(n)$ (sum, length).

- $T(n) = T(n-1) + \mathcal{O}(n) \Rightarrow \mathcal{O}(n^2)$ (insertion sort, naive reverse).
- $T(n) = 2T(n-1) + \mathcal{O}(1) \Rightarrow \mathcal{O}(2^n)$ (repeated recursive call).
- $T(n) = 2T(n/2) + \mathcal{O}(n) \Rightarrow \mathcal{O}(n \log n)$ (mergesort, balanced quicksort).
- $T(n) = T(n/2) + \mathcal{O}(1) \Rightarrow \mathcal{O}(\log n)$ (search in a balanced BST).

Example 15.2 (Why repeated calls are exponential)**.**

```
(define (bad-max lon)
  (cond [(empty? (rest lon)) (first lon)]
        [(> (first lon) (bad-max (rest lon))) (first lon)]
        [else (bad-max (rest lon))]))
```

On an increasing list, the test fails at every level, so both calls happen: $T(n) = 2T(n-1) + c$, giving $T(n) \approx c\,2^n$. A list of 30 elements needs about a billion steps. The `local` version is linear.

15.2 A short history of computing

15.2.1 Before electronic computers

- **Euclid** (c. 300 BCE) gave one of the oldest known algorithms (gcd).
- **al-Khwarizmi** (9th century) wrote on systematic procedures for arithmetic and algebra; the word "algorithm" derives from his name.
- **Charles Babbage** designed the Difference Engine and the programmable Analytical Engine (1830s–40s), never completed in his lifetime.
- **Ada Lovelace** wrote notes on the Analytical Engine including what is considered the first published program (computing Bernoulli numbers), and foresaw that such machines could manipulate symbols, not just numbers.

15.2.2 Foundations: what can be computed?

- **David Hilbert** (1928) posed the *Entscheidungsproblem*: is there a mechanical procedure to decide whether any mathematical statement is provable?
- **Kurt Gödel** (1931) showed that any consistent formal system strong enough for arithmetic contains true but unprovable statements (incompleteness).
- **Alonzo Church** (1936) introduced the **lambda calculus**, a model of computation based purely on function definition and application — the direct ancestor of `lambda` in Racket — and used it to show the Entscheidungsproblem is unsolvable.
- **Alan Turing** (1936) independently answered the same question with **Turing machines** and the undecidability of the halting problem. The Church–Turing thesis states that these models capture exactly what is "effectively computable".

The halting problem connects to Chapter 13: there is no general algorithm that decides whether an arbitrary function terminates on a given input, which is why termination arguments must be made case by case.

15.2.3 Electronic computers and programming languages

- 1940s: electronic machines (Colossus, ENIAC); **John von Neumann**'s stored-program architecture, where programs and data share memory — the basis of almost all modern computers and of the imperative programming style.
- 1950s: **Grace Hopper** built early compilers; **John Backus** led FORTRAN (1957), the first widely used high-level language.
- 1958: **John McCarthy** created **Lisp**, based on lambda calculus: lists, recursion, functions as data, garbage collection.
- 1975: **Scheme** (Sussman and Steele), a small, lexically scoped Lisp dialect; later used in the textbook *Structure and Interpretation of Computer Programs*.
- 1990s–2000s: the PLT group (Felleisen, Findler, Flatt, Krishnamurthi) built DrScheme and the *How to Design Programs* curriculum, on which CS 135 is based; renamed Racket in 2010.
- Functional ideas (lambda, map/filter/reduce, immutability) are now mainstream in Python, JavaScript, Java, C++, Haskell, OCaml, Scala, Rust, and in large-scale systems such as MapReduce.

15.3 What CS 135 did not cover

- **Mutation and state**: `set!`, mutable structures, sequencing with `begin`. These break the simple substitution model; CS 136 introduces them along with a memory model.
- **Input/output** and interaction with the outside world.
- **Arrays** and constant-time indexed access.
- **Low-level memory** and how programs actually execute on hardware.

15.4 What's next

- **CS 136** (Elementary Algorithm Design and Data Abstraction) starts in full Racket with mutation, then moves to **C**: memory, pointers, arrays, modules, ADTs, and efficiency.
- **CS 245**/**CS 240**/**CS 341**: logic, data structures and algorithms, building on recursion, trees, graphs, and big-O.
- **CS 241** and **CS 442**-level courses revisit the semantics and implementation of languages, including functional ones.

The design recipe's habits — think about data first, write contracts and examples before code, test thoroughly — carry over to every language.

15.5 Summary

- Efficiency is measured by growth of step counts with input size; use big-O and recurrences.
- Avoid repeated recursive calls and quadratic `append` patterns.
- Lambda calculus (Church) and Turing machines founded computability; Lisp brought lambda to programming, leading to Scheme and Racket.
- CS 136 adds mutation, memory, and C.